

Large Scale Combat System

Documentation

Version 3.4

Support: <https://discord.gg/uUfQuUWPCR> · Topsongames@gmail.com

Contents

Contents	2
1. Introduction.....	6
1.1 Key features.....	6
1.2 What is new in version 3.1	6
2. Support & Community.....	7
3. Quickstart	7
3.1 Requirements and import	7
3.2 Open the demo scene	7
3.3 Default controls.....	7
3.4 Your first battle.....	8
4. How LSCS is put together	10
4.1 The five layers.....	10
4.2 Data flow of a single attack	10
4.3 Layers and tags you must set up	11
5. ScriptableObjects – the data foundation	11
5.1 UnitTypeSO.....	11
5.2 DamageModifierSO	12
5.3 Combat Rulesets (plug & play damage models).....	13
StandardCombatRulesetSO – modern HP system.....	13
RomeOneHPCombatRulesetSO – Rome: Total War 1 style.....	13
5.4 UnitDataSO – the unit blueprint.....	14
General info.....	14
Combat stats.....	15
Morale settings.....	15
Animations	16
Formation settings	16
Cohesion settings	16
Detection settings	16
Behaviour	17
5.5 CombatBehaviourSO – the soldier brain	17
Settings shared by all behaviours.....	17
The compatibility matrix	17
MeleeCombatSO	18
ArcherCombatSO	19

MusketeerCombatSO	19
CavalryCombatSO and CavalrySingleCombatSO	19
ElephantCombatSO	20
HorseArcherCombatSO and HorseArcherSingleCombatSO	20
5.6 CombatPlacementSO – the formation brain in combat	21
Boid settings	21
Enemy interaction	21
Building settings	21
5.7 RangedWeaponSO – projectiles and trajectories.....	22
5.8 VisualizerSO – in-game debug overlays.....	22
5.9 EffectSO and the morale system	22
6. Building units and formations	23
6.1 The Unit prefab.....	23
Components on the root	23
Child objects	23
Waypoint references on Unit.cs.....	24
6.2 Mixed units inside one formation	24
6.3 Choosing an animation path.....	25
Which AnimatorLink do I need?	25
Filling in the mesh animation slots.....	26
Animator ↔ GPU parent switching (hybrid mode).....	26
6.4 Animation events	27
6.5 Projectiles	27
6.6 The Formation prefab.....	27
Root object	28
Formation collider child.....	28
Adding the soldiers.....	28
6.7 Army setup (ArmySO).....	29
7. Scene setup and managers.....	30
7.1 World bounds	30
7.2 GameManager	30
7.3 FormationController	31
Game mode	31
Masks and team	31
Highlighting	31
Formation settings	32

7.4 UIManager.....	33
7.5 ArmySpawner and SpawnPoint	34
7.6 TargetManager	34
7.7 Optional scene components.....	34
7.8 Mobile input	35
8. The AI system	35
8.1 The three building blocks	35
8.2 AICommander settings	36
Initial advance phase	36
8.3 Field tactics.....	37
A recommended field strategy	38
8.4 The BattleGrid (optional).....	38
Setting it up	38
Debugging the grid	39
BattleGridUnitSettingsSO – how each unit sees the field	39
8.5 Formation-level grid pathing	40
8.6 Autobattles (AI vs AI).....	40
9. The siege system	41
9.1 SiegeManager	41
9.2 Destructible gates.....	42
9.3 Capture zones (victory points)	43
Choke points.....	43
9.4 Smart defence: zones and sensors	43
ThreatSensor	43
DefensiveZone	43
AttackerZone	44
9.5 Siege tactics	44
10. Mesh animation system (performance)	45
10.1 Scene setup	45
10.2 Baking workflow	45
Step 1 – prepare the model.....	45
Step 2 – bake	46
Step 3 – configure the generated assets	46
10.3 The MeshAnimator component	47
10.4 Attachments	47
10.5 Hybrid LOD	48

11. Audio, flags and camera	48
11.1 Dynamic audio	48
11.2 Formation flags.....	48
11.3 Camera controller.....	48
12. UI, minimap and siege HUD.....	49
12.1 Siege global UI (balance of power).....	49
12.2 Siege zone UI	49
12.3 Minimap	49
13. Custom behaviour and API	49
13.1 Custom CombatBehaviourSO	49
13.2 Custom CombatPlacementSO	50
13.3 Custom AITacticSO	51
13.4 Custom RangedWeaponSO	51
13.5 Custom VisualizerSO.....	51
13.6 Formation.cs – useful members	51
13.7 Unit.cs – useful members.....	52
13.8 FormationController.cs	53
14. Troubleshooting	53
15. Component reference	54
15.1 Managers and controllers	54
15.2 Formation and movement	55
15.3 Unit and combat.....	55
15.4 Animation	56
15.5 AI.....	57
15.6 Army, siege and world.....	58
15.7 UI and visualisation	58

1. Introduction

Thank you for purchasing the **Large Scale Combat System (LSCS)**.

LSCS is a high-performance battle framework for Unity, inspired by genre classics such as *Total War*. Instead of controlling individual soldiers, the player commands cohesive **Formations**. The package ships with a complete combat framework, player controls, a modular tactical AI, a siege system and a set of job-based managers that keep large battles running smoothly.

1.1 Key features

- **High-performance core** – movement, cohesion, rotation and target acquisition run through Unity's **C# Job System** and the **Burst Compiler**.
- **GPU mesh animation** – an included animation baker plus instancing manager renders thousands of animated soldiers without SkinnedMeshRenderers. A hybrid LOD mode keeps real Animators for close-up units.
- **Modular combat behaviours** – melee, archers, musketeers, cavalry, horse archers and elephants, all defined as ScriptableObjects.
- **Swappable combat rulesets** – choose a modern HP-based damage system or a Rome 1 style two-roll "1 HP" system without touching any code.
- **Tactical AI** – a Commander AI built from Strategies and Tactics: battle lines, reserves, flanking, harassment, archer target selection, and a full siege behaviour set.
- **Optional BattleGrid** – an influence-map layer that lets the AI read danger and opportunity on the battlefield and path around threats.
- **Siege warfare** – destructible gates, walkable walls, capture zones, threat sensors and choke points.
- **Morale system** – morale, generals, aura effects and routing formations that flee to map exits.
- **Deep customisation** – every unit, weapon, tactic and visualiser is a ScriptableObject you can extend.

Video tutorials: <https://www.youtube.com/playlist?list=PL8lcLvsNB36Lg70MLPKL8Q1YkP9UQBuxA>

1.2 What is new in version 3.4

If you are upgrading from an earlier version, these are the additions you should read about first.

Feature	Where to read
BattleGrid influence map for the AI. Optional, and currently field battles only .	Chapter 8.5
Combat Rulesets – Standard HP system and Rome 1 "1 HP" system, plugged into the DamageModifierSO.	Chapter 5.3
Musketeer combat behaviour with rank fire, kneeling front row and the MusketSO ranged weapon.	Chapter 5.5 / 5.7
Horse Archers with 360° firing and kiting, in a two-mesh and a single-mesh variant.	Chapter 5.5 / 6.3

Feature	Where to read
Cavalry (Single Mesh) behaviour for rider and horse baked into one mesh.	Chapter 5.5 / 6.3
New AI tactics: Army Battle Line, Reserve & Flank Guard, Horse Archer Harass.	Chapter 8.4

2. Support & Community

Have a question, found a bug, or want to show what you are building? We are happy to help.

- **Discord (fastest support):** <https://discord.gg/uUfQuUWPCR>
- **E-mail:** Topsongames@gmail.com

When reporting a problem, please include your Unity version, render pipeline, whether you are using baked mesh animation or Unity Animators, and a screenshot of the affected prefab or ScriptableObject. That usually cuts the turnaround time in half.

3. Quickstart

3.1 Requirements and import

1. LSCS uses the **new Input System**. If your project still relies on the old input manager, set *Project Settings* → *Player* → *Active Input Handling* to **Both**.
2. A **baked NavMesh** is required in every battle scene. Units are NavMeshAgents.
3. Import the package through the Unity Package Manager.
4. If Unity asks to enable the new Input System backend, accept and let the editor restart.

3.2 Open the demo scene

Go to *TopsonGames* → *Large Scale Combat System* → *Scene* → *Battle Demo* and press Play. The demo spawns both armies and starts the enemy AI after a short delay.

3.3 Default controls

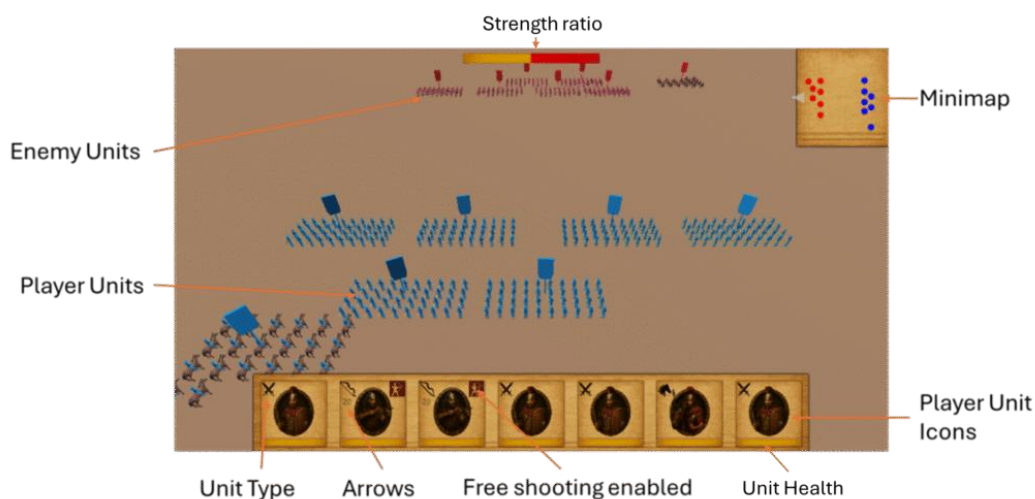
Input	Action
WASD / Mouse wheel	Move and zoom the camera
Left click	Select a formation (click the soldiers, or the unit card in the UI)
Shift + Left click	Add to / remove from the current selection
Left click & drag	Box-select multiple formations
Right click on ground	Move the selected formations there
Right click & drag	Set position and facing / width (drag out the line)
Right click on an enemy	Attack that specific formation

Input	Action
Spacebar	Show waypoints, paths and detection visualisers

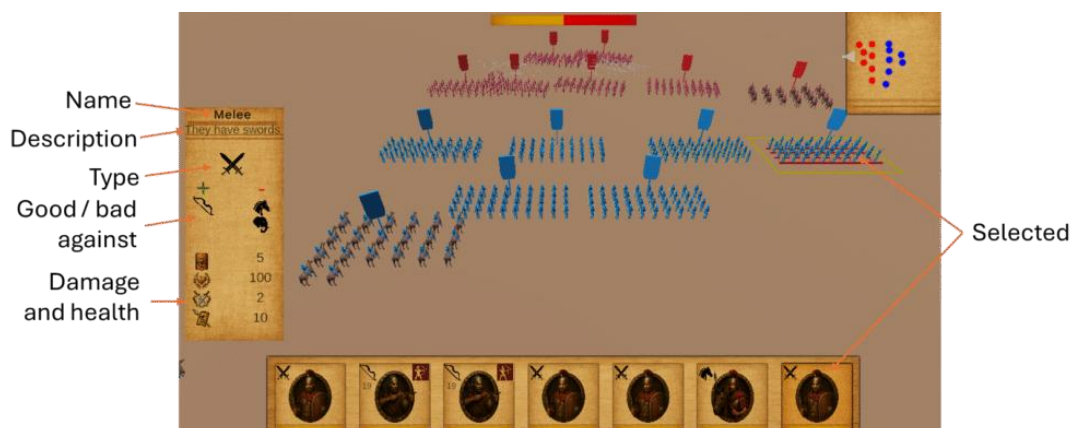
All bindings live in *TopsonGames* → *Large Scale Combat System* → *Input System* → *Input System Actions*. You can rebind everything there without touching code.

3.4 Your first battle

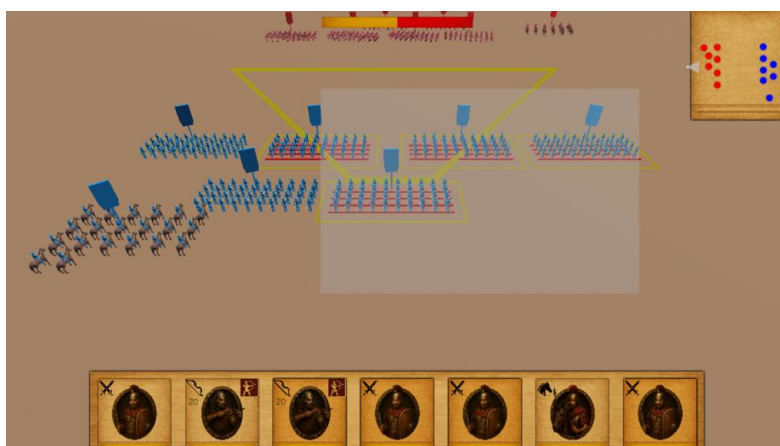
1. Press Play and wait for both armies to spawn.



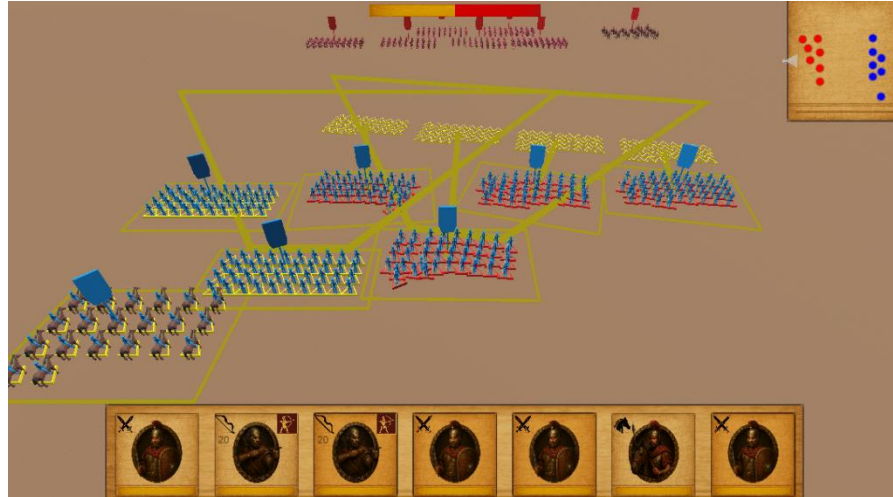
2. Left-click one of your formations. Its indicators, detection range and unit card light up.



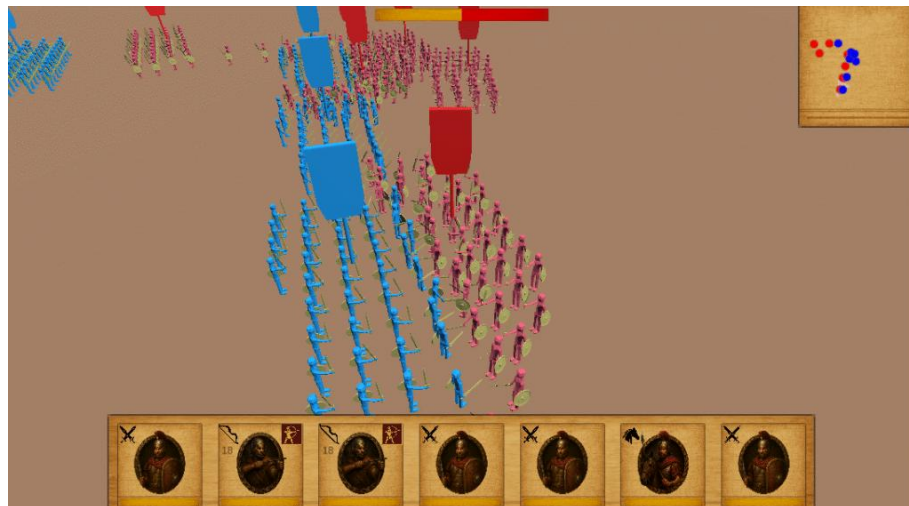
3. Drag a selection box to grab several formations at once.



4. Right-click on the ground and **hold and drag** to place the formations with a new width and facing.



5. The enemy AI advances after a few seconds. As soon as the formations touch each other's engagement triggers, combat starts automatically.



Tip: Hold **Space** during the battle. Seeing the detection trapezoids and paths is the quickest way to understand what the system is actually doing.

4. How LSCS is put together

Before you start building your own units, it helps to know which object owns which decision. Almost every support question we receive comes down to a value being edited on the wrong layer.

4.1 The five layers

Layer	Object	Responsibility
Army	ArmySO + ArmySpawner	Which unit types exist in this army, how many troops each has, and where they spawn.
Group	Formation (prefab)	Owns a list of Units. Holds state (Idle, MovingToWaypoint, Engaged, Fleeing), morale, formation width, waypoints and the enemy it is committed to.
Individual	Unit (prefab)	One soldier. Owns the NavMeshAgent, health (through DestructibleTarget), weapons and the link to the animation system. It is a middleman – it holds almost no logic of its own.
Behaviour	UnitDataSO → CombatBehaviourSO + CombatPlacementSO	All combat logic. What a soldier does when idle, moving or fighting, and how the group repositions while engaged.
Command	FormationController (player) / AICommander (AI)	Issues high-level orders to formations. Never talks to individual soldiers.

The rule of thumb: **stats and logic live in ScriptableObjects, references live in prefabs, orders come from the controllers**. If you want a unit to behave differently, you almost always edit a ScriptableObject, not a script.

4.2 Data flow of a single attack

1. FormationCollider detects an enemy EngagementTrigger and the formation switches to **Engaged**.
2. The formation's CombatPlacementSO moves each unit's waypoint into the fighting line.
3. The unit's CombatBehaviourSO.TickCombat() counts down the attack cooldown and triggers the attack animation through the AnimatorLink.
4. The animation fires the **OnAttack** animation event (pass integer 1 for ranged attacks), which the RelayAnimationEvents component forwards to Unit.Attack().
5. Unit.Attack() asks DamageModifierSO for the raw damage (base damage, armour, armour penetration, rock-paper-scissors modifier).
6. The active **Combat Ruleset** turns that raw damage into the final result (flanking, elevation, shields, or a Rome-style connect/lethality roll).
7. DestructibleTarget applies the result. On death, the formation loses a unit and morale is recalculated.

Important: Step 4 is the step people forget. **If your attack animation has no OnAttack event, your unit will play the animation and deal no damage at all.** See chapter 6.5.

4.3 Layers and tags you must set up

Name	Type	Used for
Unit	Layer + Tag	Every unit prefab. The FormationController uses this layer for selection, and the target jobs use it for enemy detection.
Collider	Layer	The formation's detection collider (FormationCollider) and the EngagementTrigger.
Ground	Layer	Walkable terrain. Used by the FormationController to place waypoints. The Default layer works too.
WallZone	Layer	Only for sieges. Assigned to DefensiveZone objects so the player can snap formations onto walls.
Shield	Tag	Optional shield collider on the unit.

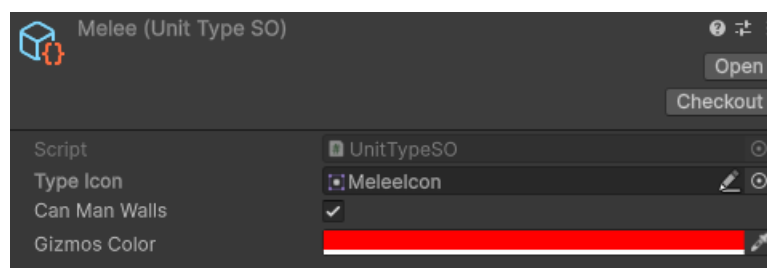
Warning: The **Ground Mask** and the **Unit Mask** on the FormationController must be different layers. If they are the same, selection silently stops working.

5. ScriptableObjects – the data foundation

The heart of LSCS is its modular ScriptableObject structure. Instead of hard-coding values, you define behaviour and properties in reusable asset files. All of them are created through the right-click menu in the Project window.

5.1 UnitTypeSO

The most basic building block. A UnitTypeSO is a **label** that defines a category, for example *Spearmen*, *Archers* or *Heavy Cavalry*. It carries almost no data itself, but the damage table, the AI tactics, the spawn points, the siege zones and the BattleGrid all identify units by it.



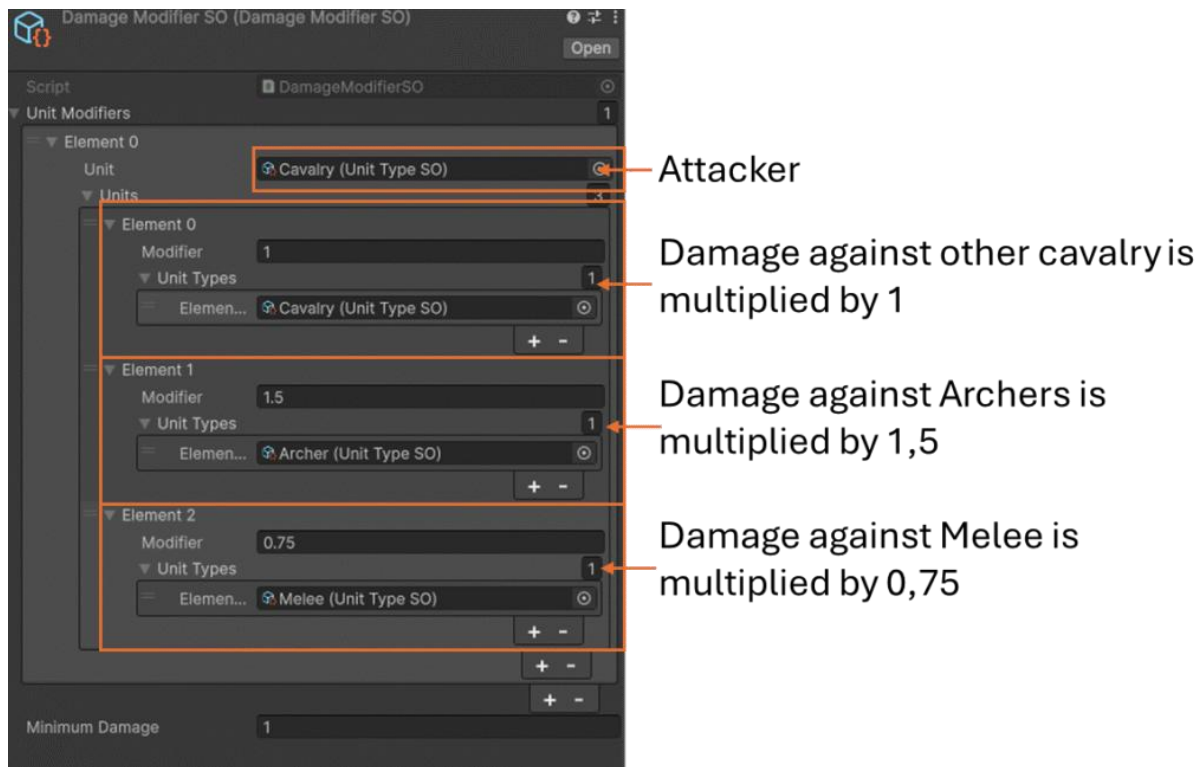
Create: Right-click → Create → TopsonGames → UnitType → UnitType

Property	Meaning
Type Icon	Sprite used in the UI.
Gizmos Colour	Colour for debug visualisation in the scene view.
Can Man Walls	Whether this type may be placed on defensive walls. True for infantry and archers, False for cavalry and elephants.

Tip: Create your UnitTypes **first**, before anything else. Almost every other asset references them, and renaming them later is painless while re-assigning them everywhere is not.

5.2 DamageModifierSO

This asset defines the rock-paper-scissors balance of your game. You normally need exactly **one** in the project, and it is assigned to the GameManager.



Create: Right-click → Create → TopsonGames → DamageModifier → DamageModifier

Property	Meaning
Active Combat Ruleset	The damage model used by the whole game. See 5.3. If you leave this empty, the standard behaviour of the units applies.
Unit Modifiers → Attacker	The UnitType dealing the damage.
Unit Modifiers → Modifier	Multiplier, e.g. 1.5 = 150 % damage, 0.6 = 60 % damage.
Unit Modifiers → Unit Types	The list of UnitTypes that receive this multiplier.
Minimum Damage	Damage floor. Guarantees that even a weak attacker scratches heavy armour.
Flanking Modifier	Multiplier applied when an attack comes from the side or rear (e.g. 1.5).

The base damage formula, before the ruleset is applied, is:

```
effectiveArmor    = max(0, defenderArmor - attackerArmorPenetration)
damageAfterArmor  = max(minimumDamage, attackerBaseDamage - effectiveArmor)
rawDamage         = unitModifier(attackerType, defenderType) * damageAfterArmor
```

The AI also reads this table. GetGoodAgainst() and GetBadAgainst() drive the unit tooltips, and the BattleGrid weights its danger maps with these modifiers. **A well-filled damage table therefore makes the AI smarter**, because it starts to prefer favourable matchups.

5.3 Combat Rulesets (plug & play damage models)

New in this version: the way damage is finally applied is itself a ScriptableObject. Drop the ruleset you want into the **Active Combat Ruleset** field of your DamageModifierSO. Nothing else has to change.

StandardCombatRulesetSO – modern HP system

The default. Damage is subtracted from hit points, with optional tactical multipliers. Leave every multiplier at 1.0 if you want the plain basic system.

Create: Right-click → Create → TopsonGames → Combat Rulesets → Standard HP System

Property	Meaning
Elevation Threshold	Height difference in metres from which the elevation bonus/malus applies.
High Ground Damage Multiplier	Damage multiplier for attacks coming from above (1.0 = no bonus).
Low Ground Damage Multiplier	Damage multiplier for attacks coming from below (1.0 = no malus).
Facing Angle Threshold	Angle in degrees within which an attack counts as frontal. Inside this cone the shield can block.
Flanked Damage Multiplier	Extra multiplier when the target is hit from the flank or rear (1.0 = off).

RomeOneHPCombatRulesetSO – Rome: Total War 1 style

A faithful rebuild of the classic two-roll model. Every strike rolls **twice**:

1. **Connect** = $\text{baseConnectChance} * \text{connectGrowthPerPoint}^{(\text{Attack} - \text{Defence})}$, clamped. Low base value, exponential growth, so equal stats produce very few kills while a stat advantage snowballs.
2. **Lethality** = a separate roll per weapon (sword high, arrow low).
3. Only if **both** rolls succeed does the target lose one hit point.

The result is the grinding melee of the original game, where battles are decided by morale rather than raw kill speed.

Create: Right-click → Create → TopsonGames → Combat Rulesets → Rome One 1-HP System

Property	Meaning
Base Connect Chance	Connect chance when Attack equals Defence. Original values are roughly 0.08–0.15.
Connect Growth Per Point	Factor per point of (Attack – Defence). 1.2 ≈ +20 % per point, 1.1 is grindier.
Attack Defense Diff Clamp	Caps the stat difference so nobody ever reaches 0 % or 100 %.
Min / Max Connect Chance	Hard floor and ceiling for the connect roll.
Melee / Ranged Lethality Fallback	Used when a unit does not define its own <code>meleeWeaponLethality</code> .
Treat All Units As Single Hitpoint	On: a lethal hit removes the whole health bar, so every soldier is effectively 1 HP regardless of the HP value. Off: only <code>hitPointDamage</code> is deducted, which gives you true multi-HP units (1 for a soldier, 5 for an elephant).
Hit Point Damage	Damage per lethal hit when the option above is off.
Ranged Damage Scaling	Converts a projectile's armour-adjusted damage into a connect chance. Higher = less lethal arrows.

Note: The Rome ruleset uses three extra fields at the bottom of the **UnitDataSO**: *Melee Attack Skill*, *Melee Defense Skill* and *Melee Weapon Lethality*. They are ignored by the standard ruleset, so it is safe to leave them at 0 unless you use this system. Remember that the actual kill rate is *chance per strike × strike frequency* – if a battle resolves too quickly, raise the `cooldown` in the combat behaviour before touching the lethality values.

5.4 UnitDataSO – the unit blueprint

The centrepiece. One `UnitDataSO` defines a complete unit type from A to Z, and it is the asset you will edit most often.

Create: Right-click → Create → TopsonGames → Unit Data → UnitDataSO

General info

Property	Meaning
Unit Icon	Sprite for the UI.
Description	Short text shown in the hover UI.
Formation Prefab	The formation prefab that gets spawned for this unit type. This is what the <code>ArmySpawner</code> instantiates.
Cost	Resource cost. Reserved for your own economy code, no built-in function.

Combat stats

Property	Meaning
Max Health	Total hit points of a single soldier.
Armor	Flat reduction of incoming damage.
Base Damage	Damage per successful hit.
Armor Penetration	Ignores this much of the target's armour.
Charge Damage Multiplier	Bonus damage while the formation's charge bonus is active (e.g. 1.5).
Charge Damage Duration	How many seconds the charge bonus lasts after the formation makes contact.
Shield Melee Block Chance	0.0–1.0 chance to block a melee hit that arrives from the front.
Shield Arrow Block Chance	0.0–1.0 chance to block a projectile. -1 means "use the melee value" .
Arrows	Ammunition for ranged units. This counts volleys of the formation , not arrows per soldier. Set 0 for pure melee units.
Can Recieve Knockback	Whether cavalry and elephant charges can physically push this unit. Turn off for elephants and other heavy units.
Unit Type	Reference to the UnitTypeSO.
Attackable Target Types	What this unit is allowed to attack. <i>*Add Gate and Building for siege-capable units, otherwise they will stand in front of a gate and do nothing.*</i>

Morale settings

Property	Meaning
Base Morale	Starting morale of the formation without any external influence.
Is General	Marks this unit as the army's general. Its death applies a morale hit to the whole army. The system does not check whether an army contains more than one general – that is up to you.
Morale Penalty Per Health Loss	Morale lost for every 10 % of the formation's total health that is gone.
General Death Impact	How hard this formation is hit when the general dies.
Effects On Other Formations	EffectSO assets this formation radiates to nearby friendly formations. Put the <i>General Nearby</i> boost here for general units.
Effect Radius	Radius in world units in which those effects are applied.

When morale reaches 0, the formation switches to **Fleeing**, runs to the nearest MapExit and its soldiers are removed once they touch the exit trigger. To visualise a general's aura in-game, add the *General Visualizer* prefab as a child of the formation and register it in the formation's **Visualizers** list.

Animations

Property	Meaning
Idle Animations	How many idle variants this unit has. If greater than 1, the behaviour randomly switches between them for visual variety.
Attack Animations	How many attack variants exist. The behaviour picks a random index per attack.

Formation settings

Property	Meaning
Formation Width	Default number of soldiers per row.
Formation Spacing	Distance between soldiers, e.g. 1.7.
Formation Move Speed	Walking speed of the formation.
Formation Turn Speed	How fast the formation rotates as a block.
Unit Rotation Speed	How fast an individual soldier turns.

Cohesion settings

These control how strictly soldiers stick to their assigned slot while marching. They are evaluated inside a Burst job.

Property	Meaning
Unit Catch Up Factor	How strongly a lagging soldier accelerates. Higher = catches up faster.
Unit Slow Down Factor	How strongly a leading soldier brakes. Higher = tighter line.
Max Speed Multiplier	Speed ceiling while catching up (1.5 = 150 % of normal speed).
Min Speed Multiplier	Speed floor while braking (0.7 = 70 %).
Stopping Distance	Distance from its waypoint at which a soldier stops.

Detection settings

Property	Meaning
Collider Overhang	How far the formation's detection collider extends beyond the soldiers.
Max Collider Size	Upper limit for the detection box (X/Z).

Property	Meaning
Collider Height	Vertical size of the logic collider. Set this to about 3.0 in siege maps so units on the ground do not engage enemies standing on top of a wall.
Center Smoothing Speed	How quickly the calculated formation centre follows the soldiers. Lower = smoother, less jitter.

Behaviour

Property	Meaning
Combat Behaviour	The CombatBehaviourSO that drives every individual soldier. See 5.5.
Placement Behaviour	The CombatPlacementSO that repositions the whole formation while engaged. See 5.6.

Warning: The Combat Behaviour and the animation link on your unit prefab are **not** interchangeable. Cavalry, horse archer and musketeer behaviours require specific AnimatorLink components. See the compatibility matrix in 5.5 before you pick one.

5.5 CombatBehaviourSO – the soldier brain

A CombatBehaviourSO defines what a single soldier does in each state: Idle, Moving and Fighting. It is a ScriptableObject, so one asset is shared by every soldier of that type – never store per-soldier state in it.

Settings shared by all behaviours

Property	Meaning
Unit Detection Timer	Seconds between enemy scans for an individual soldier.
Formation Detection Timer	Seconds between enemy scans for the whole formation.
Engagement Break Distance	Distance at which combat is exited. Reserved for future development.
Attack Range	Distance at which a melee attack is performed.
Cooldown	Seconds between attacks. This is your main lever for how fast a melee resolves.
Raise Shield Time	How long the shield stays raised after arrows are detected.
Switch Animation Timer	Seconds between idle-animation re-rolls.
Switch Animation Probability	Chance (0–1) that the idle animation actually changes on a re-roll.

The compatibility matrix

This is the table to bookmark. Each combat behaviour talks to the animation system through a specific AnimatorLink class. Picking the wrong pair is the single most common setup mistake – it typically shows up as

soldiers that slide without animating, horses that never move their legs, or a `NullReferenceException` the moment the first unit attacks.

Combat Behaviour (UnitDataSO)	Required AnimatorLink on the visuals	Notes
MeleeCombatSO	AnimatorLinkMesh (baked / hybrid)	Standard infantry. Any of the two generic links works.
ArcherCombatSO	AnimatorLinkMesh	Foot archers with a trapezoidal detection area. Switches to melee when out of ammo.
MusketeerCombatSO	AnimatorLinkMeshMusketeer	Needs the extra <code>AttackRangedKneeling</code> clip and <code>SetRowInFormation</code> support for rank fire.
ElephantCombatSO	AnimatorLinkMesh	Single-body mount, so no separate horse link is required.
CavalryCombatSO	AnimatorLinkMeshCavalry	Rider and horse are two separate meshes / animators. The behaviour casts to <code>AnimatorLinkCavalry</code> and will throw if the link is a plain <code>AnimatorLink</code> .
CavalrySingleCombatSO	AnimatorLinkMeshCavalrySingle	Rider and horse baked into one single mesh. Cheaper, but supports only Idle, Walk, Attack and Death.
HorseArcherCombatSO	AnimatorLinkMeshHorseArchers	Two-mesh mount with 360° firing. Inherits from <code>AnimatorLinkMeshCavalry</code> .
HorseArcherSingleCombatSO	AnimatorLinkMeshHorseArchersSingle	Single-mesh mount with 360° firing, separate idle and walk firing sets. Inherits from <code>AnimatorLinkMeshCavalrySingle</code> .

Rule of thumb: The word **Single** in a name always means *rider and mount are one baked mesh*. Everything without *Single* keeps the mount as its own mesh with its own animator, which costs more but lets the horse animate independently (for example a dying rider on a still-running horse).

MeleeCombatSO

Standard infantry. Beyond the shared settings it only adds **Wander Radius**, the small random offset a soldier keeps around its waypoint so lines do not look mathematically perfect.

Create: TopsonGames → Combat Behaviours → Melee

ArcherCombatSO

Create: TopsonGames → Combat Behaviours → ArcherCombat

Property	Meaning
Archer Detection Range	Maximum distance at which the formation looks for a target.
Archer Inner Angle	Smaller, inner angle of the trapezoidal firing area.
Archer Outer Angle	Larger, outer angle of the trapezoid.
Archer Front Offset	How far the trapezoid is pushed forward from the formation centre.
Archer Detection Timer	Seconds between target scans.
Time Between Shots	Seconds between volleys.
Projectile Speed	Speed of the arrow. Also used to lead moving targets.

Archers fire as a **formation volley**: the formation picks one enemy formation, and every soldier then picks an individual target inside it. Each volley consumes one point of Arrows. When ammunition runs out, or the formation is engaged in melee, it switches to `CombatState.Melee` and draws its melee weapon. Add the *Archer Range Visualizer* prefab to see the trapezoid in-game.

MusketeerCombatSO

Line infantry with rank fire. It uses the same trapezoidal detection as the archer plus two extra settings.

Create: TopsonGames → Combat Behaviours → Musketeer Combat

Property	Meaning
Max Firing Rows	How many rows may fire at the same time. 2 = front and second rank. Soldiers in deeper rows hold their fire.
First Row Kneels	If enabled, the very first row plays the kneeling firing animation and the rows behind it stand.

Unlike archers, musketeers spread their fire: each soldier picks a **random** living enemy inside the target formation, which distributes casualties across the enemy block instead of deleting one file of men. Pair this behaviour with the `MusketSO` ranged weapon (5.7) and the *Musketeer Visualizer*.

CavalryCombatSO and CavalrySingleCombatSO

Identical settings, different animation requirements (see the matrix above).

Create: TopsonGames → Combat Behaviours → Cavalry — or — Cavalry (Single Mesh)

Property	Meaning
Knockback Force	Physical impulse applied to the victim.
Knockback Duration	How long the victim stays knocked back before it re-joins the NavMesh.
Knockback Radius	Radius used in Area mode.
Knockback Damage Multiplier	Damage multiplier applied on the impact hit.
Knockback Behaviour	SingleUnit – only the direct target is thrown. Area – everyone inside the radius is thrown, explosion style.
Wander Radius	Random offset around the waypoint while fighting.

Note: Knockback only affects units whose UnitDataSO has **Can Recieve Knockback** enabled. Give heavy or braced units immunity there instead of removing the cavalry's knockback.

ElephantCombatSO

Same knockback fields as cavalry, but defaults to **Area** knockback so the animal tramples several soldiers at once. It is also included as a worked example of how to extend the base class.

HorseArcherCombatSO and HorseArcherSingleCombatSO

Mounted skirmishers that can shoot in any direction while riding, including the classic Parthian shot backwards over the saddle.

Create: TopsonGames → Combat Behaviours → Horse Archer — or — Horse Archer (Single Mesh)

Property	Meaning
Archer Detection Range	Firing range. The AI harassment tactic reads this value to decide its standoff distance.
Archer Detection Timer	Seconds between target scans.
Time Between Shots	Seconds between shots.
Knockback Force / Duration	Applied when the horse archer is forced into melee.
Wander Radius	Random offset around the waypoint.

The animation link splits the full circle into **eight directional firing clips** (F, FR, R, BR, B, BL, L, FL) and rotates the shootPointParent transform on the Y axis so the arrow really leaves the bow in the direction of the target. The single-mesh variant additionally has a separate set of clips for firing while standing and firing while riding.

Tip: Horse archers only look right if their formation keeps its distance. Use the **Horse Archer Harass AI** tactic (8.4) or they will happily ride into a spear wall.

5.6 CombatPlacementSO – the formation brain in combat

While a CombatBehaviourSO controls one soldier, the CombatPlacementSO controls how the **whole formation** holds or adjusts its shape once it meets an enemy. The default implementation uses boids logic to create a dynamic push-and-pull along the front line. An older, purely algorithmic version is also included if you prefer a rigid line.

Boid settings

Property	Meaning
Neighbor Radius	Radius in which a soldier considers other friendly soldiers as neighbours.
Separation Radius	Minimum distance a soldier tries to keep from friendly neighbours.
Cohesion Force	Pull towards the average position of the neighbours.
Separation Force	Push away from neighbours that are too close.
Alignment Force	How strongly a soldier matches the average heading of its neighbours.
Formation Cohesion Force	Pull back to its own designated slot in the formation grid.

Enemy interaction

Property	Meaning
Enemy Attraction Force	Magnetic pull towards the enemy.
Enemy Separation Radius	Minimum distance kept from individual enemy soldiers.
Enemy Separation Force	Push that prevents soldiers from walking into enemy bodies.
Engagement Distance	Range at which the formation breaks its parade shape and actively engages.
Arrival Distance	Distance at which a soldier starts slowing down for a smooth stop.
Stop Distance To Enemy	Final distance a soldier tries to keep from its opponent.
Enemy Frontline Sample Size	How many of the closest enemies are averaged to find the enemy front line. Larger = smoother, less reactive.

Building settings

Property	Meaning
Building Attack Distance	Desired distance from the surface of a building or gate.
Min Attack Point Spacing	Minimum distance between two attack slots around a building.
NavMesh Sample Radius	Search radius for NavMesh.SamplePosition / FindClosestEdge.
Max Building Attack Points	Maximum number of attack slots generated per building.

5.7 RangedWeaponSO – projectiles and trajectories

A RangedWeaponSO decides how a projectile is launched: its trajectory, speed and accuracy. Assign it to the RangedWeapon component on the weapon model of your unit prefab.

Asset	Created via	Behaviour
IBowSO	TopsonGames → Ranged Weapon → Bow	High-arcing ballistic volley. Solves for the launch angle so arrows drop onto the target.
MusketSO	TopsonGames → Ranged Weapon → MusketSO	Flat, fast shot. Predicts the target's movement and adds a random <i>Inaccuracy Offset</i> per shot – set it to (0,0,0) for perfect accuracy.

Both read `projectileSpeed` and pull projectiles from the weapon's object pool via `GetNextArrow()`. To add a crossbow, a javelin or a catapult, inherit from RangedWeaponSO and override `Attack()` – see chapter 13.

5.8 VisualizerSO – in-game debug overlays

Visualisers draw tactical information into the world using a LineRenderer. Prefabs live under *TopsonGames* → *Large Scale Combat System* → *Prefabs* → *Visualizers*. Drag the one you need as a child of a formation and add it to the formation's **Visualizers** list.

Visualizer	Shows
PathVisualizerSO	The path the formation will take to its destination.
MeleeDetectionVisualizerSO	Position, size and rotation of the formation's detection collider.
ArcherVisualizerSO	The archer's trapezoidal firing area.
MusketeerVisualizerSO	The musketeer's firing area.
HorseArcherDetectionVisualizerSO	The horse archer's 360° firing radius.
GeneralVisualizerSO	The aura radius in which a general applies its morale effect.

Which visualisers appear, and when, is controlled by the **Highlighting** section of the FormationController (7.3).

5.9 EffectSO and the morale system

An EffectSO is a named status a formation can carry, for example *General Nearby*, *Flanked*, *Under Fire* or *Shooting*. Effects are applied in two ways:

- **Radiated:** a formation lists them under *Effects On Other Formations* in its UnitDataSO and applies them to friendlies within *Effect Radius*.
- **Situational:** the combat code raises them itself, for example when a formation is being shot at.

Effects are displayed as icons in the UI (the icon set is configured on the UIManager) and feed into the morale calculation. Morale itself is recalculated from base morale, health losses, active effects and the death of the general. At 0 morale the formation routs.

6. Building units and formations

This chapter walks through the actual GameObjects. LSCS keeps a strict split between the **logic level** (the Unit GameObject) and the **visual level** (a child object holding the model, the animator and the weapons). Keeping that split intact is what allows the system to swap the entire animation backend at runtime.

Recommended order of work:

5. Create the UnitTypeSO assets.
6. Create the UnitDataSO and fill in the stats (you can leave the behaviour empty for now).
7. Build the Unit prefab.
8. Set up the animation system on the Unit prefab and pick a matching Combat Behaviour.
9. Build the Formation prefab and fill it with Unit prefabs.
10. Point the UnitDataSO's *Formation Prefab* field at that formation prefab.
11. Add the unit to an ArmySO.

6.1 The Unit prefab

Create an empty GameObject, for example RomanLegionary_Unit, and set its **Layer and Tag** to **'Unit'**.

Components on the root

Component	Required	Purpose
Unit.cs	Yes	The central middleman. Holds every reference the behaviours need.
NavMeshAgent	Yes	Pathfinding and local avoidance.
CapsuleCollider	Yes	Physical representation of the soldier.
DestructibleTarget.cs	Yes	Health, team ID and death events. Set its Target Type to 'Unit' . Without it the unit cannot take damage at all.
UnitAudio	Optional	Footsteps, swings, death sounds. Triggered by animation events.
AnimatorLOD	Optional	Lets the AnimatorLODManager throttle the Animator update rate by distance.

Child objects

Child	Contains
Visuals	The 3D model, the Animator and/or the baked MeshAnimator, the matching AnimatorLink component, and RelayAnimationEvents (or RelayAnimationEventsMesh).
Melee Weapon	The sword, spear or axe model. Assign it to Unit.MeleeWeapon.

Child	Contains
Ranged Weapon	The bow or musket model with the <code>RangedWeapon.cs</code> script. Needs a <i>ShootPoint</i> child transform where projectiles spawn, and a <code>RangedWeaponSO</code> asset. Assign it to <code>Unit.RangedWeapon</code> .
Shield	Optional. A <code>BoxCollider</code> set to <i>IsTrigger</i> with the tag <i>Shield</i> . Assign the collider to <code>Unit.shieldCollider</code> .
Waypoints	Drag in the <i>Waypoint</i> prefab from <i>TopsonGames</i> → <i>Large Scale Combat System</i> → <i>Prefabs</i> → <i>Indicators and Waypoint</i> and assign its four children to the Unit script (see below).

Important: The visuals – Animator, SkinnedMeshRenderer and MeshAnimator – **must be a child of the Unit GameObject**, never on the root itself. The animation LOD system enables and disables these objects, and it would switch off the logic with them.

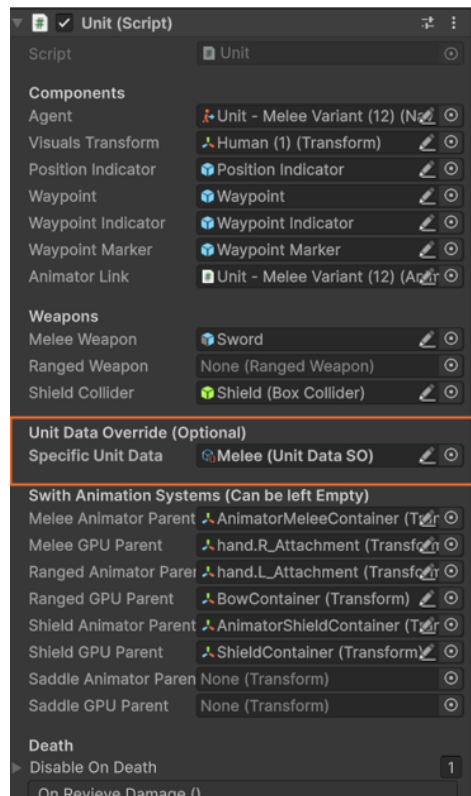
Waypoint references on Unit.cs

Field	Meaning
Position Indicator	Shows the soldier's current position on hover or selection.
Waypoint	The actual target transform the soldier walks to. This is the object the placement behaviour moves.
Waypoint Indicator	Preview of the new slot while the player drags a move order.
Waypoint Marker	The visible marker at the destination.
Visuals Transform	The visuals child. Used for rotation and death handling.
Disable On Death	Scripts that should be switched off when the soldier dies.

6.2 Mixed units inside one formation

A formation may contain several unit types, for example a block of spearmen with archers behind them. To give a soldier its own stats and behaviour, assign a `UnitDataSO` to the **Specific Unit Data** field of that Unit prefab inside the formation. If the field is empty, the formation's own `UnitDataSO` is used.

Note: If you mix ranged and melee soldiers in one formation, the **base type – the UnitDataSO on the Formation script – has to be the ranged one**. Otherwise the formation never enters the archer combat state and the ranged soldiers stay silent. An example lives in *Prefabs* → *Formations* → *Mixed*.



6.3 Choosing an animation path

LSCS supports three animation setups. They are not exclusive – the hybrid mode is a combination of the first two and is what the demo scene uses.

Path	Component on visuals	When to use it
Unity Animator	AnimatorLink / AnimatorLinkCavalry	Small battles, cinematics, prototyping. Full Mecanim features, highest cost per soldier.
Baked GPU mesh	MeshAnimator + AnimatorLinkMesh...	Thousands of soldiers. Vertex-baked animation drawn with GPU instancing, no bones at runtime.
Hybrid LOD (recommended)	Both, managed by AnimatorLinkLODManager	Real Animators for units close to the camera, baked meshes for everyone else. Best of both.

In hybrid mode the AnimatorLinkLODManager calls EnableAnimator() and DisableAnimator() on every registered link based on its **Cull Distance**. The AnimatorLink then enables one renderer and disables the other, so from the player's point of view nothing changes.

Which AnimatorLink do I need?

Use the compatibility matrix in chapter 5.5. In short:

- Infantry and foot archers → AnimatorLinkMesh
- Musketeers → AnimatorLinkMeshMusketeer (adds the kneeling fire clip and rank awareness)
- Cavalry with a separate horse mesh → AnimatorLinkMeshCavalry
- Cavalry baked as one mesh → AnimatorLinkMeshCavalrySingle
- Horse archers, two meshes → AnimatorLinkMeshHorseArchers

- Horse archers, one mesh → AnimatorLinkMeshHorseArchersSingle

Filling in the mesh animation slots

Every AnimatorLinkMesh... component exposes a list of MeshAnimation assets produced by the baker. Which slots exist depends on the variant:

Component	Animation slots
AnimatorLinkMesh	Idle, Walk, Attack, AttackRanged, RaiseShield, Death, Knockback
AnimatorLinkMeshMusketeer	The same, plus AttackRangedKneeling for the front rank
AnimatorLinkMeshCavalry	Rider: Idle, Walk, Attack, RaiseShield, Death, Knockback — Horse: HorseIdle, HorseWalk, HorseDeath (two MeshAnimators)
AnimatorLinkMeshCavalrySingle	Idle, Walk, Attack, Death only – the rider and mount are one mesh
AnimatorLinkMeshHorseArchers	Everything from AnimatorLinkMeshCavalry, plus eight directional clips: AttackRangedF, FR, R, BR, B, BL, L, FL
AnimatorLinkMeshHorseArchersSingle	Everything from AnimatorLinkMeshCavalrySingle, plus two sets of eight directional clips – one for firing while standing, one for firing while riding

Horse archer links also need the **Shoot Point Parent** transform assigned – usually the bow or the hand bone holder. The system rotates it on the Y axis so the projectile leaves in the direction the rider is actually aiming.

Animator ↔ GPU parent switching (hybrid mode)

Baked meshes have no bones, so weapons cannot be parented to a hand bone while a unit is in GPU mode. LSCS solves this with paired parent transforms on the Unit script. When the LOD manager switches modes, the weapon is re-parented automatically.

Field pair on Unit.cs	Assign
Melee Animator Parent / Melee GPU Parent	The hand bone of the rigged model / the corresponding attachment point on the baked mesh.
Ranged Animator Parent / Ranged GPU Parent	Same for the bow or musket.
Shield Animator Parent / Shield GPU Parent	Same for the shield.
Saddle Animator Parent / Saddle GPU Parent	Same for the rider on a mounted unit.

Note: These fields may be left empty. If you only use Unity Animators, or only baked meshes, nothing needs switching. They matter exclusively for the hybrid LOD mode. The saddle pair is only used by the two-mesh cavalry links.

6.4 Animation events

Animation events are the bridge between the animation and the combat logic. The component that receives them sits on the visuals object: `RelayAnimationEvents` for Unity Animator clips, `RelayAnimationEventsMesh` for baked clips. Both expose the same functions and both need their **Unit** and (optionally) **UnitAudio** references assigned.

Event	Effect
<code>OnAttack(int type)</code>	The most important one. 0 = melee, 1 = ranged. Deals the damage or fires the projectile. Without it, no damage is ever applied.
<code>OnEnableMeleeWeapon()</code>	Shows the melee weapon and hides the ranged one.
<code>OnEnableRangedWeapon()</code>	Shows the ranged weapon and hides the melee one.
<code>OnEnableShield()</code> / <code>OnDisableShield()</code>	Enables or disables the shield collider, which drives the block chance.
<code>OnWalk()</code>	Footstep sound.
<code>OnSwing()</code>	Weapon swing sound.
<code>OnDeath()</code>	Death sound.
<code>OnCheer()</code>	Cheer sound.

For Unity clips, add the events in the Animation window as usual. For **baked** clips, select the generated `MeshAnimation` asset in the Project window and add the events in its inspector: function name, int value, and normalized time between 0 and 1.

6.5 Projectiles

Every projectile is its own prefab.

12. Create the projectile model.
13. Add `Projectile.cs`. *Lifetime* controls how long it survives, *Collider Time* is a short delay before its damage becomes active so it cannot hit the shooter.
14. Add a **Collider** set to *IsTrigger* and a **Rigidbody**.
15. Place the projectile prefab as a hidden child of the unit's **ShootPoint** so the pool can be initialised.

Warning: Never Instantiate projectiles at runtime. `RangedWeapon.cs` maintains an object pool, and bypassing it will cost you far more performance than the projectiles are worth.

6.6 The Formation prefab

The formation prefab is the container – the "captain" – that turns a group of unit prefabs into one commandable body. Create one formation prefab per unit type you want to field.

Root object

Component / field	Setup
Formation.cs	The brain of the group. Manages its unit children, calculates the grid, receives orders and owns the state machine.
Rigidbody	Required. Set <i>Is Kinematic</i> to true. Without a Rigidbody on the parent, the child trigger colliders do not report reliably.
Team ID	Which side this formation belongs to. Only formations whose Team ID matches the FormationController can be commanded by the player. When spawning through the GameManager this is set automatically.
Unit Data	The UnitDataSO created earlier.
Visualizers	List of the Visualizer children you added (path, detection, general aura ...).
Engagement Trigger	Reference to the EngagementTrigger child.
Check Interval	How often, in seconds, the formation checks for nearby formations to apply aura effects. Raise it if you have very many formations.

Formation collider child

1. Create a child GameObject and set its **Layer** to `'Collider'`.
2. Add a BoxCollider with *IsTrigger* enabled.
3. Add FormationCollider.cs – this is the detection volume that finds enemies. Its *Max Engagement Height Difference* (default 3.0) makes it ignore enemies far above or below, which is what keeps ground troops from fighting men on a wall.
4. Add EngagementTrigger.cs – this is what **other** formations detect. Think of FormationCollider as the eyes and EngagementTrigger as the body.
5. Link both scripts to the parent Formation script.

Adding the soldiers

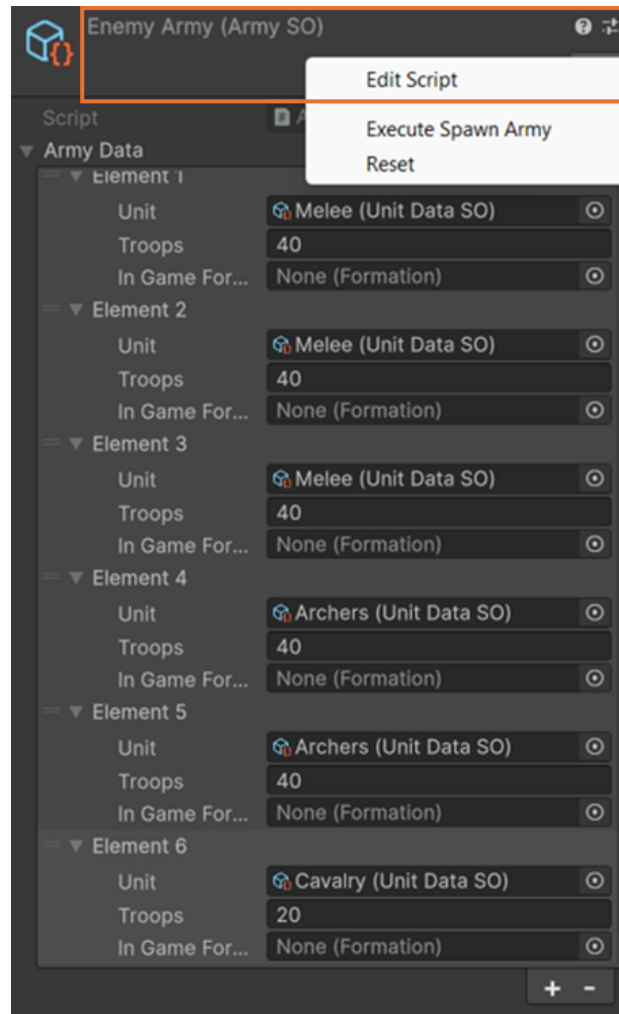
1. Drag your Unit prefab in as a child.
2. Duplicate it (Ctrl+D) until you reach the desired count, for example 40.
3. Press **Create Formation** in the Formation inspector to arrange them into the grid.

Finally, go back to your UnitDataSO and assign this formation prefab to its **Formation Prefab** field. That is what the spawner instantiates.

6.7 Army setup (ArmySO)

An ArmySO is the roster of one side. **Create:** Right-click → Create → TopsonGames → Army

1. Add one element per unit type.
2. Assign the UnitDataSO and the number of **Troops** – this is the number of individual soldiers, which the spawner divides into formations.
3. Right-click the component header → **Execute Spawn Army** to refill the roster.



If *Reset Army After Playing* is enabled on the GameManager, the troop counts return to their starting values when you stop playing. Disable it if you want a campaign where losses persist between battles.

Field	Meaning
On Player Win / On Enemy Win	Hook up your victory and defeat screens.
On Player / Enemy General Died	Fired when a general falls. Useful for announcements.
Reset Army After Playing	If true, troop counts in the ArmySO are restored after play mode ends.
Building Attack Range Buffer	Important for sieges. About 1.5. Extra tolerance that lets units attack gates and buildings even though the NavMeshObstacle stops them short of the collider.

7.3 FormationController

The singleton that processes all player input, manages the selection and issues orders to player formations. It sits on the Manager object.

Game mode

Field	Meaning
Current Game Mode	PlayerVsAI – normal play. Spectator_AIvsAI – both sides are AI-controlled and the player can only watch and inspect. See 8.7.
Player AI Commander	Only for spectator mode: the AICommander that takes over the player's army.

Masks and team

Field	Meaning
Ground Mask	The physics layer of your walkable ground. Tells the controller where waypoints may be placed. Default works.
Unit Mask	The layer your units are on. Must be a different layer than the Ground Mask or selection will not work. Unit is recommended.
Wall Zone Mask	Siege only. The layer of your DefensiveZone objects. Enables snapping a formation onto a wall.
Team ID	Which formations the player may command. Set to 0 to control all formations with Team ID 0.

Highlighting

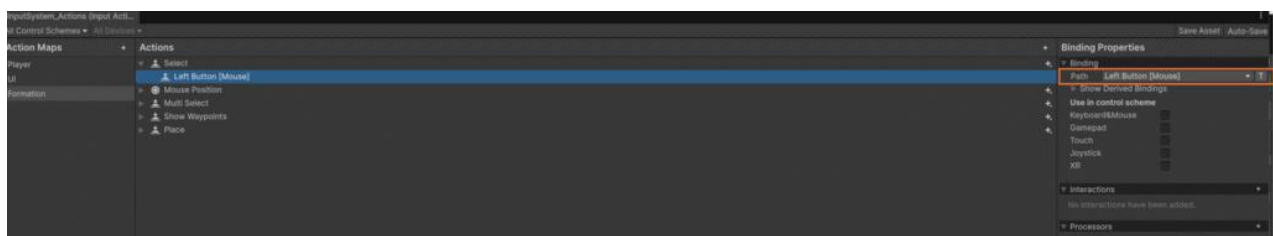
Field	Meaning
Show Indicator On Highlight	Shows selection indicators when hovering a formation.
Show Enemy Indicator	Also show hover indicators on enemy formations.
Hover Hide Delay	Small delay before a hover indicator fades, to prevent flickering.

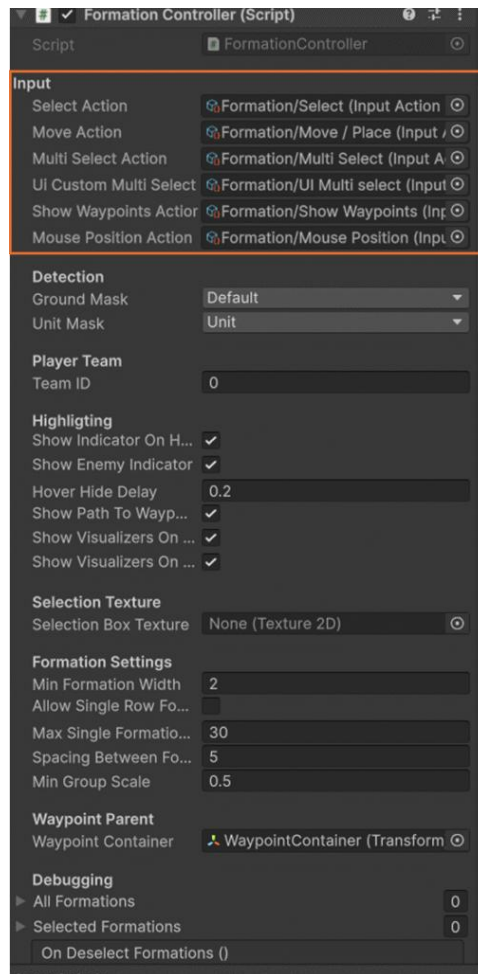
Field	Meaning
Show Path To Waypoint	Holding the Show Waypoints key also draws each formation's path.
Show Visualizers On Highlight	Show a formation's visualisers on hover. Combined with <i>Show Enemy Indicator</i> , enemy visualisers appear too.
Show Visualizers On Select	Show visualisers when a formation is selected.
Selection Texture	Texture for the drag-selection box. A semi-transparent default is used if empty.

Formation settings

Field	Meaning
Min Formation Width	Smallest number of soldiers per row the player may drag to.
Allow Single Row Formations	If enabled the player can stretch a formation into one long line. If disabled, the maximum width is one less than the soldier count, forcing at least two ranks.
Max Single Formation World Width	Maximum width in world units when dragging a single formation. Also used to derive the maximum width for multi-formation drags.
Spacing Between Formations	Gap inserted between formations when several are dragged together.
Min Group Scale	Minimum visual size of a formation, so a badly mauled unit stays visible and selectable.
Waypoint Container	Parent object that all waypoints are attached to. Keeps the hierarchy clean.
Disengage Cooldown Duration	How long a formation waits before it may auto-attack again after receiving a manual move order. Prevents units from immediately re-engaging the enemy you just pulled them away from.

Input can be customized at the InputSystem actions which can be found at *TopsonGames\Large Scale Combat System\Input System\Input System Actions* Here you can assign new Inputs.

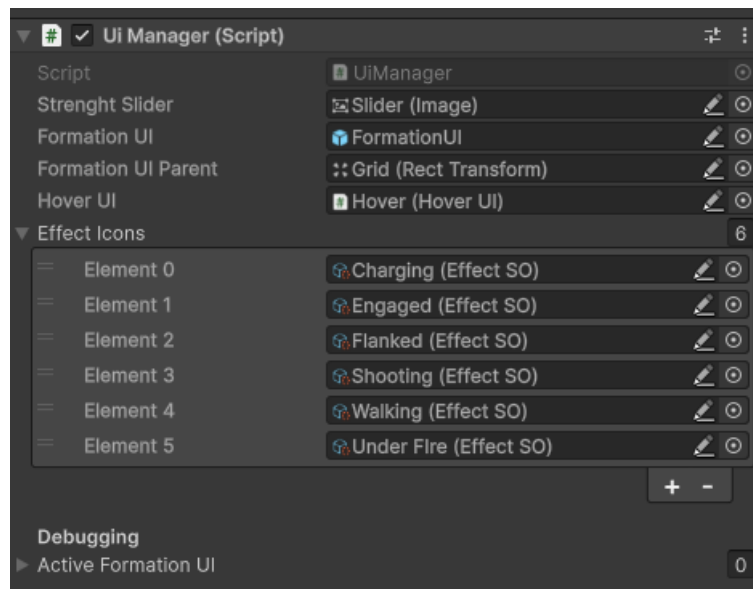




7.4 UIManager

Handles everything on screen except the minimap. We recommend using the provided canvas and swapping the sprites rather than rebuilding it.

Field	Meaning
Strength Slider	Visualises player strength against enemy strength.
Formation UI	Prefab spawned for every player formation. Lets the player select a formation by clicking its card.
Formation UI Parent	Where those cards are spawned.
Hover UI	Panel showing formation stats on hover. Leave empty to disable it.
Effect Icons	The icon set for all effects a formation can carry.



7.5 ArmySpawner and SpawnPoint

Each army needs its own ArmySpawner GameObject.

1. Create an empty GameObject and add `ArmySpawner.cs`.
2. Create empty GameObjects at the positions where troops should appear and add `SpawnPoint.cs` to each.
3. Add those spawn points to the ArmySpawner's list.
4. Press **Snap To NavMesh** to make sure every point is on valid ground.

SpawnPoint field	Meaning
Allowed Unit Types	Restricts which UnitTypes may spawn here, for example cavalry only on the flanks. Leave empty to allow everything.
Inverse Forward Rotation	Rotates the spawned formation by 180° on the Y axis. Use it when your formations face the wrong way.

7.6 TargetManager

A performance helper that keeps sorted lists of every attackable target in the scene – units, gates and buildings – so the target-acquisition jobs do not have to search the whole scene. It needs no configuration, but it must be present in the scene.

7.7 Optional scene components

Component	Purpose
MapExit	Trigger volume at the edge of the map. Routing formations run here and their soldiers are removed on contact. A field battle should always have at least one , otherwise fleeing units run forever.

Component	Purpose
NoPlacementZone	Put this on a collider to forbid the player from placing formations there – water, cliffs, the enemy deployment area. Draws a red gizmo in the editor.
StairTrigger	Marks a staircase so units correctly change level between ground and wall in siege maps.
TowerController	A defensive tower that automatically attacks enemies of a given team within range. Configure <i>Enemy Team ID</i> , <i>Attack Range</i> , <i>Attack Damage</i> , <i>Attack Cooldown</i> and the attack type. Optionally spawns a particle effect at the <i>Attack Origin</i> and appears on the minimap.
FormationCollider / EngagementTrigger	Covered in 6.6 – they belong to the formation prefab, not the scene.

7.8 Mobile input

MobileInputManager provides a touch control scheme on top of the normal FormationController. Add it to your Manager object, assign the FormationController and the CameraController, and hook up the UI buttons.

Field	Meaning
Selection Mode Toggle Button / Selected Color	Switches between camera panning and box selection, and the colour used while selection mode is active.
Deselect Button	Clears the current selection.
Selection Box Visual	The Image used to draw the drag rectangle.
Long Press Time	Seconds a finger must rest before a long press is registered.
Pan / Zoom / Rotation Sensitivity	Touch sensitivity for the camera.
Drag Threshold	How many pixels a finger must travel before it counts as a drag instead of a tap.
Touch Position / Touch Contact Action	Input System action references for the touch device.

8. The AI system

The AI is driven by the **AICommander**. It works the way a human player does: every few seconds it looks at the battlefield, scores a list of possible tactics, and executes the best one. Everything it can do is data, not code – you build an AI personality by assembling ScriptableObjects.

8.1 The three building blocks

Asset	Role
AICommander (component)	One per army. Holds the strategies, decides when to think and keeps track of which formations are still free to receive orders.

Asset	Role
AIStrategySO	A named list of tactics – effectively an AI personality, for example "Aggressive Field Army" or "Siege Defender".
AITacticSO	One concrete manoeuvre. Each tactic implements Evaluate(), which returns a score, and Execute(), which issues the orders.

On every think tick the commander evaluates all tactics in its current strategy and runs them in order of score. A tactic that has claimed formations marks them as busy, so lower-scoring tactics only get the units nobody else needed. This is why the **score values matter**: a reserve tactic scoring 90 will take its share of troops before the battle line tactic at 82 lays out the front.

8.2 AICommander settings

Field	Meaning
Default Strategy	Used for open field battles.
Siege Attacker Strategy	Used automatically when a SiegeManager is in the scene and this commander's team is the attacker.
Siege Defender Strategy	The same for the defending team.
Think Interval	Seconds between decisions, e.g. 3.0. Lower is more reactive but more expensive.
Team ID	Which army this commander controls.

The commander detects the siege case on its own: if a SiegeManager exists it compares its Team ID against the manager's attacker and defender IDs and picks the matching strategy. If the corresponding siege strategy is empty it falls back to the default one and logs a warning.

Initial advance phase

Freshly spawned armies used to hesitate, because at spawn distance no tactic has a strong opinion. The initial advance phase fixes that by simply marching forward for a while before normal decision making starts.

Field	Meaning
Enable Initial Advance	Turns the phase on.
Initial Advance Unit Types	Which unit types march forward. Usually your line infantry, not the cavalry.
Initial Delay	Seconds the AI waits after the battle starts before it moves at all.
Advance Duration	How long the stubborn march lasts before the regular tactics take over.
Advance Distance	How far ahead the target point is placed during this phase.

8.3 Field tactics

Tactic	What it does	Key settings
Tactic_ArmyBattleLine (score 82)	The modern replacement for <i>Establish Front Line</i> . Treats all melee formations as one line rather than individuals: it computes a shared battle direction, deploys the whole line at a standoff distance, then charges together. Targets are matched by index, so left faces left and formations never cross paths.	lineSpacing, standoffDistance, chargeRange, slotTolerance, allowNeighbourSwap, neighbourSwapThreshold
Tactic_ReserveFlankGuard (score 90)	Gives the army a sense of self-preservation. Holds a percentage of the melee formations behind the centre as a reserve, watches for enemies entering the flanks or rear, and sends the nearest reserve to intercept. Runs before the battle line so it can claim its reserves first.	reserveEligibleTypes, reservePercentage, rallyDistanceBehindCenter, frontArcDegrees, flankThreatRange, criticalThreatRange
Tactic_EstablishFrontLine (score 80)	The original, simpler front line logic. Matches each melee formation against an enemy melee formation individually. Still supported – use it if you prefer looser, more chaotic engagements.	directOpponentBonus, assignmentPenalty
Tactic_FlankWithCavalry	Sends cavalry around the main clash to hit vulnerable rear targets such as archers. Uses BattleGrid pathing when available so the horses ride around spear walls instead of through them.	CavalryTypes, VulnerableTargetTypes, chanceForEarlyAttack, fallbackFlankDistance, rearFlankDistance
Tactic_HorseArcherHarass (score 85)	Kiting for mounted skirmishers. Reads the unit's own archerDetectionRange, keeps the formation at shootDistanceVariable of that range and rides away when an enemy comes closer than skirmishDistance. Attaches a SkirmisherKiteAgent to the formation automatically.	HorseArcherTypes, VulnerableTargetTypes, shootDistanceVariable, skirmishDistance
Tactic_ArcherTargetSelection	Assigns the best target to each ranged formation, weighing distance, target vulnerability, line of	VulnerableTargetTypes, vulnerabilityBonus, stickinessBonus,

Tactic	What it does	Key settings
	sight and – with the BattleGrid – how dangerous the firing position itself would be.	distancePenaltyWeight, dangerPenaltyWeight, enemyBlockingLayer

Tip: stickinessBonus on the archer tactic is worth tuning early. Without it, archers keep re-picking targets every tick and their volleys never land. A value around 20 makes them commit.

A recommended field strategy

A solid, general-purpose AIStrategySO for open battles contains, in this order:

1. Tactic_ReserveFlankGuard – keeps a reserve and answers flank attacks
2. Tactic_HorseArcherHarass – if you field horse archers
3. Tactic_ArmyBattleLine – forms and charges the main line
4. Tactic_FlankWithCavalry – sends the cavalry around
5. Tactic_ArcherTargetSelection – keeps the archers usefully employed

Remember to fill the UnitTypeSO arrays inside every tactic. A tactic whose type list is empty simply scores 0 and never runs – this is by far the most common reason for an AI that "does nothing".

8.4 The BattleGrid (optional)

Scope: The BattleGrid is **entirely optional** and is currently used for **field battles only**. Siege support is planned for a future update. Every tactic listed above works without it – the grid only makes the AI smarter about *where* it walks. If no BattleGridManager exists in the scene, all tactics fall back to direct pathing automatically.

The BattleGrid is an **influence map**: an invisible grid laid over the battlefield in which every cell carries a number. Two families of maps are generated, per team and per unit type:

- **Danger maps** – how dangerous a cell is *for this unit type*. Melee formations stamp a box or circle around their engagement volume, archers stamp a cone in their firing direction, and each shape is scaled by the damage modifier between the two types. A spear wall is therefore very red for cavalry and only mildly red for infantry.
- **Target maps** – how attractive a cell is. Enemy formations stamp a value based on how well this unit type performs against them, with an extra bonus behind the enemy formation so flanking manoeuvres are naturally preferred.

Tactics then query these maps: GetDangerMap(), GetTargetMap(), FindSafePath() and GetNearestSafePosition(). The whole grid is rebuilt in a Burst job every updateInterval, spread across a frame boundary so it does not spike.

Setting it up

1. Create an empty GameObject in the scene, e.g. BattleGrid, and add BattleGridManager.
2. Position it at the centre of your battlefield, or use *Origin Offset* to shift it.

3. Size the grid so it covers the playable area: width × cellSize by height × cellSize world units. A white wireframe box shows the coverage in the scene view.
4. Fill **Tracked Unit Types** with every UnitTypeSO the AI should reason about. **Only listed types get maps.**
5. Optionally create a BattleGridUnitSettingsSO and assign it to fine-tune how each unit type perceives danger.

Field	Meaning
Width / Height	Number of cells. 100 × 100 at cell size 2.5 covers 250 × 250 world units.
Cell Size	Size of one cell in world units. Smaller is more precise and more expensive.
Center Grid	If true, the grid is centred on the object's position instead of starting there.
Origin Offset	Manual shift of the grid origin.
Update Interval	Seconds between rebuilds. 0.5 is a good default.
Pathfinding Clearance	How many cells of margin a path keeps from dangerous cells. Higher = wider detours, and formations need the room to actually walk them.
Unit Grid Settings	Optional BattleGridUnitSettingsSO, see below.
Tracked Unit Types	Which UnitTypes get their own danger and target maps.

Debugging the grid

Set **Show Gizmos**, pick a **Debug Unit Type** and a **Debug Team ID**, then choose the mode:

- **Danger** – draws the red danger map that the chosen unit type of the chosen team perceives.
- **Targets** – draws the green attractiveness map.
- **None** – only the grid bounds.

Active AI paths are drawn as coloured lines with spheres at each waypoint, which makes it obvious why a cavalry formation chose the route it did.

Tip: If the AI takes absurd detours, your danger values are too high or the *Pathfinding Clearance* is too large. If it walks straight into archers, either the archers are not in *Tracked Unit Types* or the damage modifier between the two types is 1.0, so the grid sees no reason to care.

BattleGridUnitSettingsSO – how each unit sees the field

Create: Right-click → Create → TopsonGames → AI → Grid → Battle Grid Unit Settings

The asset holds a list of *Viewer Settings*. A **viewer** is the unit type doing the pathfinding; a **threat** is the enemy type on the field.

Field	Meaning
Viewer Type	The unit type this entry describes, e.g. Cavalry.
Default Radius / Value Multiplier	Applied to every enemy type not listed specifically.
Specific Threats → Threat Types	The enemy types this exception applies to, e.g. Spearmen.
Danger Radius Multiplier	Enlarges or shrinks the danger area of those enemies for this viewer.
Danger Value Multiplier	Raises or lowers how bad that danger feels.

A typical configuration: viewer *Cavalry*, specific threat *Spearmen* with radius 2.0 and value 3.0 – cavalry now gives spear formations a very wide berth – and specific threat *Archers* with radius 0.5 and value 0.5, so the same cavalry happily rides through arrow fire to reach them.

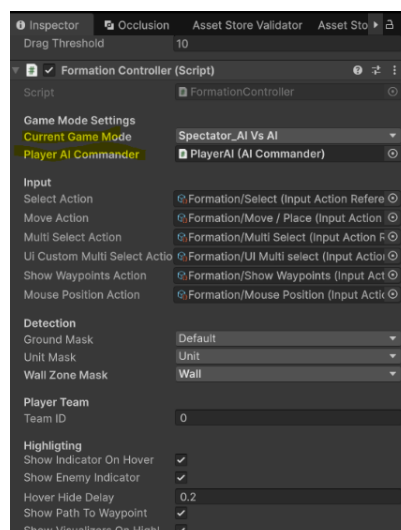
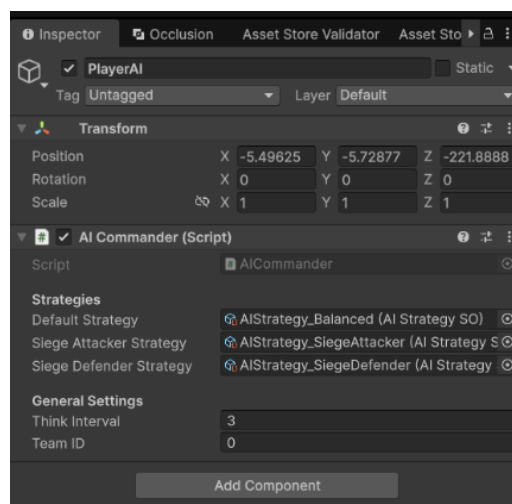
8.5 Formation-level grid pathing

Formations can also follow a grid path on their own when they have been given a custom enemy. The relevant runtime fields (useGridPath, gridWaypoints, currentGridRiskTolerance ...) are set by the tactics and are visible in the inspector during play, which is useful when debugging why a formation took a particular route.

8.6 Autobattles (AI vs AI)

1. Create a new GameObject with an AICommander component and set its **Team ID to 0** – the player's ID, or whichever ID you gave the player formations.
2. Assign the strategies you want that army to use.
3. On the Manager object, open the FormationController, set **Current Game Mode** to **Spectator_AI Vs AI** and assign the new commander to **Player AI Commander**.

Both armies are now AI-controlled. The player can still move the camera and inspect unit stats but cannot issue orders. This works in field battles and in sieges alike, and it is the fastest way to balance-test a damage table: let the same two armies fight ten times and watch which side wins.



9. The siege system

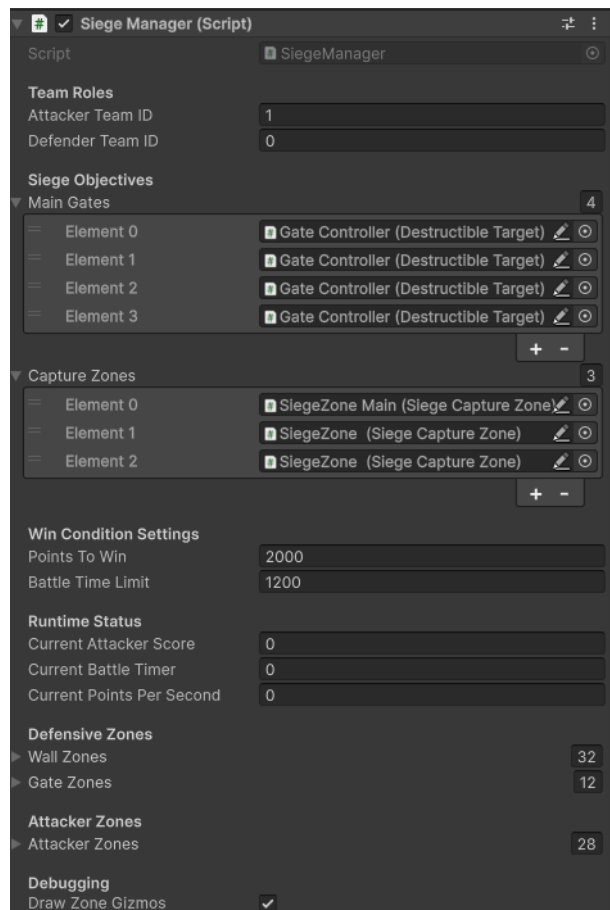
The siege system turns an open field into a fortified objective map. It is built from three kinds of object: **static structures** (gates, walls), **tactical zones** (where units should stand) and **sensors** (what the AI can perceive).

9.1 SiegeManager

Add this component to your Manager GameObject. Its mere presence switches every AICommander in the scene into siege mode.

Field	Meaning
Attacker / Defender Team ID	Who is besieging and who is holding. Usually the player is the attacker or defender depending on your mission.
Main Gates	Every DestructibleTarget that acts as a gate.
Capture Zones	Every SiegeCaptureZone in the scene.
Points To Win	Score the attacker must accumulate through capture zones to win (tug of war).
Battle Time Limit	Seconds. When the timer expires, the defender wins.
Wall Zones	All DefensiveZone objects on walls.
Gate Zones	All DefensiveZone objects behind gates.
Attacker Zones	All AttackerZone objects outside the walls.
Draw Zone Gizmos	Editor visualisation of all zones.

Warning: Zones that exist in the scene but are **not** in these lists are invisible to the AI. If your defenders ignore a wall, check the list first.



9.2 Destructible gates

A gate is a physical barrier that blocks the NavMesh and can be destroyed to open a path.

1. Create your gate GameObject.
2. Add a **NavMeshObstacle**. Enable **Carve** and disable *Carve Only Stationary*, so the NavMesh is cut while the gate stands and reconnects when it is destroyed.
3. Add **DestructibleTarget** and set its Target Type to **Gate**.
4. Add **GateController** and assign the *Intact Visuals* and *Destroyed Visuals* objects.
5. Optionally assign an **Attacker Sensor** – a **ThreatSensor** placed in front of the gate – so the AI knows how many enemies are at the door. *Defense Detection Radius* controls how far behind the gate defenders are counted.

Reminder: Units can only attack a gate if **Gate** is in their **UnitDataSO's Attackable Target Types**, and the **GameManager's Building Attack Range Buffer** is large enough (≈ 1.5) to compensate for the NavMesh carve.

9.3 Capture zones (victory points)

Strategic areas such as a town square or a gatehouse that decide the battle.

Field	Meaning
Capture Time	Seconds needed to flip the zone.
Initial Owner Team ID	Who holds it at the start – normally the defender.
Is Victory Point	Whether the zone counts as an objective at all.
Contributes To Result	Whether it feeds the global score. Lets you build zones that matter tactically but not for victory.
Priority	How valuable it is. Drives both the score rate and how strongly the AI wants it.
Choke Points	SiegeChokePoint objects marking the entrances, see below.
Tactical Approach Points	Transforms the AI uses as staging positions before entering.
Minimap Icon	Sprite shown on the minimap.
On Owner Changed / On Contested	UnityEvents you can hook your own audio or UI onto.

Units must physically stand inside the trigger collider to capture. Ownership is a tug of war: while the attacker controls a zone they gain points based on its priority; the defender holding it stops or reverses the gain.

Choke points

Without guidance, every unit ordered to a zone runs at its centre and jams in the doorway. A SiegeChokePoint marks a valid entrance.

Field	Meaning
Capacity	How many formations fit side by side – 1 for a narrow alley, 3 for a main street.
Is Attacker Entry	Whether this entrance is meant for attackers or defenders.

The capture tactics distribute formations across the available choke points instead of funnelling them all through one gap.

9.4 Smart defence: zones and sensors

ThreatSensor

A trigger collider that counts enemy units inside it, re-scanning every *Scan Interval* seconds. On its own it does nothing – its value comes from being linked to a gate or a defensive zone.

DefensiveZone

Marks a position where defenders, usually archers, should stand. The interesting part is its **dynamic priority**:

1. Every zone has a **Base Priority** – the main gate wall might be 50, a back wall 10.

2. If a ThreatSensor is linked, the zone adds that sensor's current threat level to its priority.
3. So a back wall at priority 10 that suddenly sees an approaching army jumps to priority 110, and the AI pulls troops off the quiet main gate to answer the real attack.

Field	Meaning
Linked Structure	The gate or building this zone protects. Helps the AI group its defences.
Linked Sensor	The ThreatSensor watching the ground in front of this wall.
Zone Width / Depth	Dimensions of the standing area. Match the width to the wall segment.
Base Priority	Importance without any enemies present.
Preferred Unit Types	Which unit types belong here. Add your archer types so cavalry never tries to climb a wall.
Neighbors	Adjacent zones. Fill manually, or right-click the component → Auto Find Neighbors (Distance) .

AttackerZone

The attacker's counterpart: a firing or staging position outside the walls. Place them where an archer would have a clean line onto the battlements.

Field	Meaning
Linked Structure	Which gate or building this zone is aimed at.
Zone Width / Depth	Desired formation width in world units. 0 ignores width.
Preferred Unit Types	Optional restriction on who may occupy it.
Priority	Higher means the AI prefers this position.

9.5 Siege tactics

Tactic	Side	Behaviour and key settings
Tactic_AttackGate	Attacker	Picks the best gate by weighing distance against defender strength (read from the gate's ThreatSensor) and sends GateBreakerTypes to the AttackerZones linked to it. MaxZonesToAssignPerTick limits churn.
Tactic_OccupyArcherPositions	Attacker	Moves ArcherUnitTypes into AttackerZones, preferring zones with clear line of sight to the walls. MaxArcherRange filters out positions that are too far to be useful.
Tactic_ManTheWalls	Defender	Assigns RangedWallUnits to DefensiveZones by dynamic priority, so threatened walls pull troops away from quiet ones. repositionThreshold prevents units from oscillating between two zones of similar value.

Tactic	Side	Behaviour and key settings
Tactic_DefendGate	Defender	Sends <code>MeleeGateDefenders</code> into the <code>DefensiveZones</code> behind a gate once at least <code>minAttackersForThreat</code> enemies are detected at it.
Tactic_CapturePoints	Attacker	Activates once a gate is breached. Directs <code>CaptureUnitTypes</code> to capture zones, weighing zone priority, existing assignments and enemy presence. Archers listed under <code>ArcherTypes</code> look for firing positions instead of standing in the zone. <code>sightBlockerLayers</code> is used for the line-of-sight check.
Tactic_RecapturePoints	Defender	The mirror image: retakes zones the attacker has captured, using the same weights plus a penalty for crowding the same approach point.

10. Mesh animation system (performance)

Standard `SkinnedMeshRenderers` cannot render thousands of animated soldiers. LSCS ships a GPU instancing solution with a custom animation baker: each frame of an animation is baked into its own mesh, and the instancing manager draws all soldiers sharing a mesh and material in a single call.

- Extended documentation: http://topsongames.com/?page_id=339
- Video tutorials: <https://www.youtube.com/playlist?list=PL8lclvsNB36L1gGZFEQhxNmuhUsJiFbhl>

10.1 Scene setup

1. Make sure `MeshAnimation.cs`, `MeshAnimator.cs`, `MeshInstancingManager.cs` and `AttachmentPoint.cs` are in the project, with `AnimationBaker.cs` inside an *Editor* folder.
2. Create an empty `GameObject` and add **MeshInstancingManager**.
3. Fill in the LOD settings below.

Field	Meaning
Lod Distances	Global LOD distances in ascending order, e.g. 25, 60, 120. Units beyond the last entry are culled.
Buckets Per LOD	How many animation <i>buckets</i> each LOD uses, e.g. 4, 2, 1. Soldiers in the same bucket share an animation phase, so a group appears to move in step. Fewer buckets at distance means less CPU work and is invisible from far away.

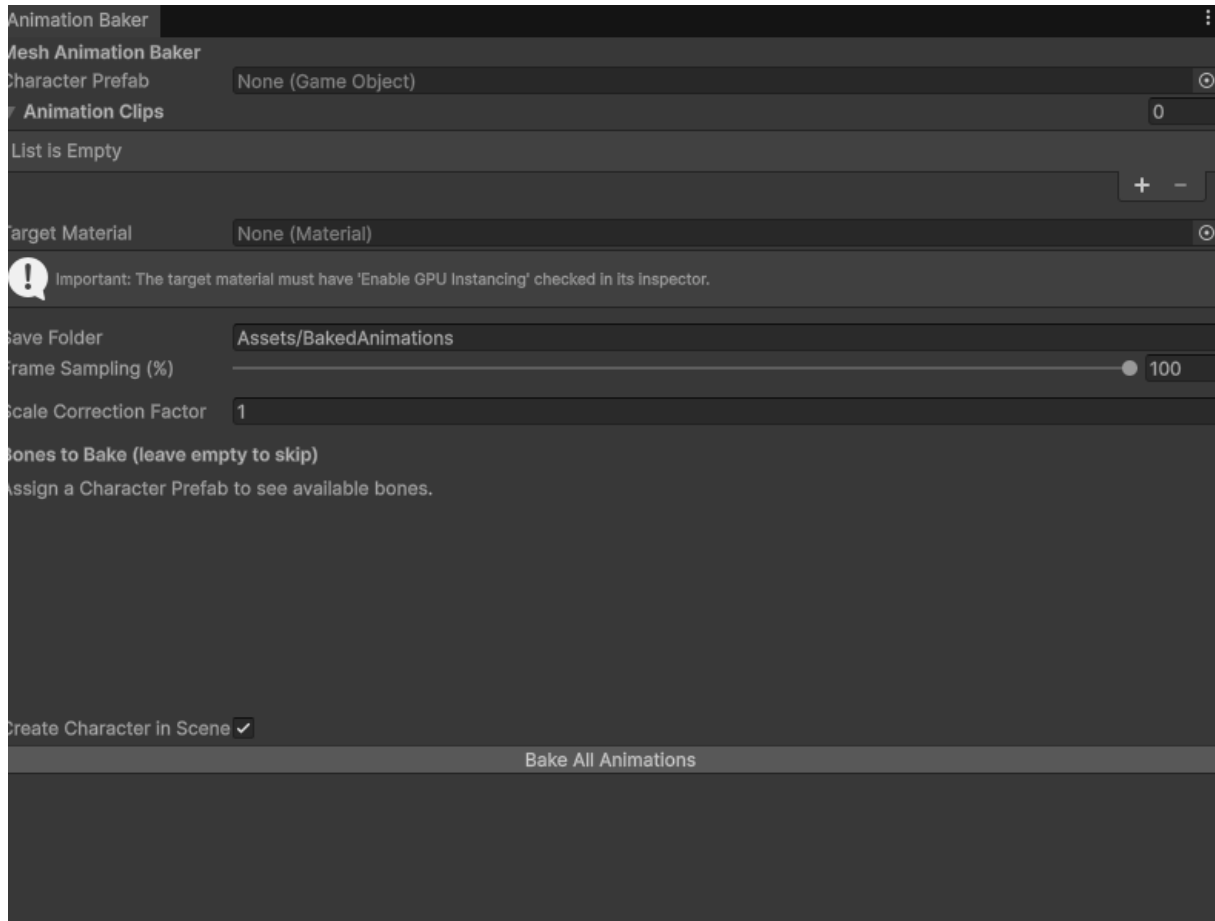
10.2 Baking workflow

Step 1 – prepare the model

Your character should use **one material**, and that material must have **Enable GPU Instancing** ticked in its inspector.

Step 2 – bake

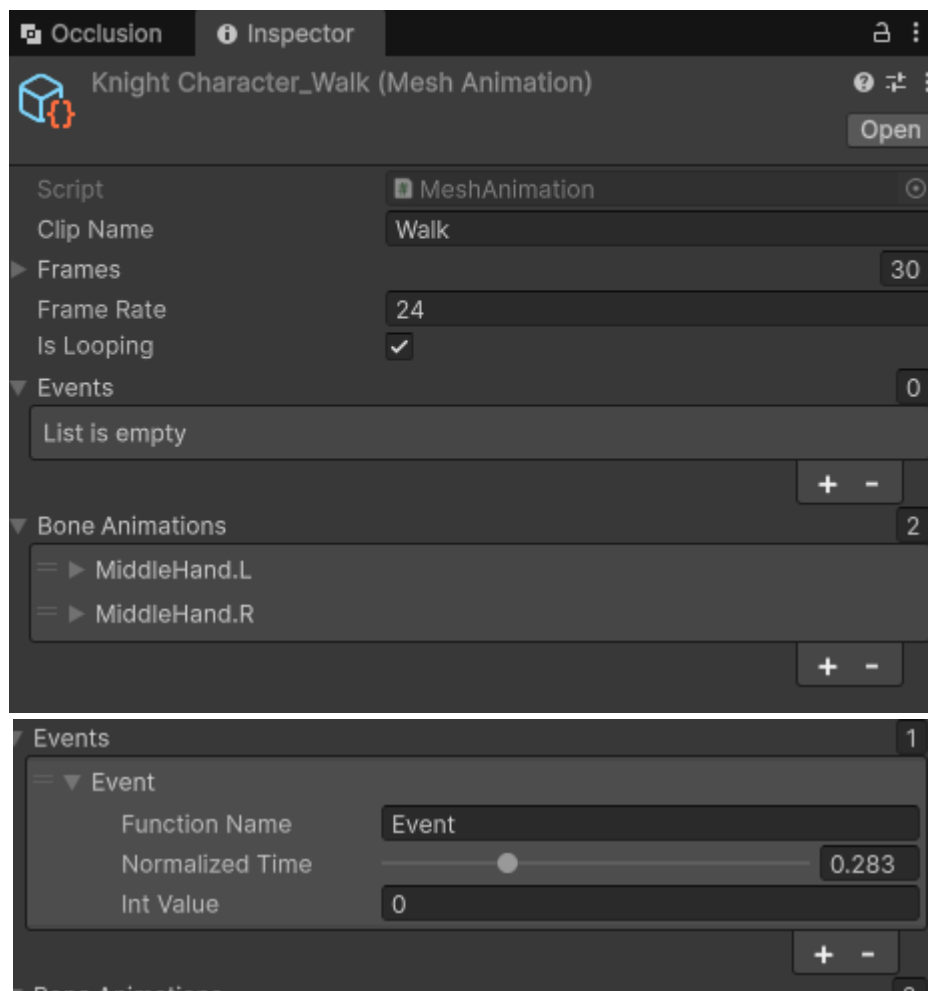
1. Open *Tools* → *TopsonGames* → *Animation Baker*.
2. **Character Prefab**: drag in your model. It must have a `SkinnedMeshRenderer`.
3. **Animation Clips**: add every clip you want – idle, walk, attack, death, and so on.
4. **Bones To Bake**: if the unit carries weapons, list the exact bone names, e.g. `mixamorig:RightHand`. These are tracked so the attachment system can glue objects to them later.
5. Press **Bake All Animations**.



Step 3 – configure the generated assets

The baker produces one `MeshAnimation` asset per clip. Select each one and set:

Field	Meaning
Clip Name	Identifier used by <code>MeshAnimator.Play(string)</code> .
Frame Rate	Playback rate of the baked frames.
Speed Multiplier	Per-clip speed correction. Useful when a baked clip plays too fast or too slow.
Is Looping	Whether the clip repeats. Turn this off for attack and death clips.
Events	Crucial for combat. Add function name, int value and normalized time (0–1). At minimum, every attack clip needs <code>OnAttack</code> with int 0 for melee or 1 for ranged.



10.3 The MeshAnimator component

Sits on the visuals of a unit and plays baked clips.

Field	Meaning
Target Mesh Filter	The MeshFilter whose mesh is swapped every frame.
Lod Levels	One list of MeshAnimations per LOD level. Level 0 is the highest quality. You can bake lower-poly versions of the same clips for distant LODs.
Play Automatically / Auto Play Clip	Plays a clip on start. If no clip is given, the first clip of LOD 0 is used.
Playback Speed	Runtime speed multiplier.
Bucket Looping Animations	Enables the bucket system for looping clips – see 10.1.
Bone Scale	Extra multiplier on baked bone positions. Should normally stay at 1.

10.4 Attachments

Baked meshes have no runtime bones, so weapons are attached through the AttachmentPoint system instead.

1. Add AttachmentPoint.cs to the weapon GameObject.

2. **Target Animator:** the unit's MeshAnimator.
3. **Bone Name:** the exact bone name you baked, e.g. `mixamorig:RightHand`.
4. **Offsets:** adjust position and rotation until the weapon sits correctly in the hand.

If you selected bones during baking, this is largely set up for you.

10.5 Hybrid LOD

Add `AnimatorLinkLODManager` to the scene and set its **Cull Distance** – the distance at which units switch from real Animators to baked GPU meshes. Every `AnimatorLinkMesh...` component registers itself automatically on enable, so there is nothing else to wire up.

A second, independent system exists for pure Animator setups: `AnimatorLODManager` plus `AnimatorLODSettings` throttle the *update rate* of distant Animators instead of replacing them. Put `AnimatorLOD` on the unit prefab to opt in.

11. Audio, flags and camera

11.1 Dynamic audio

A pooled audio system that prevents audio spam in large battles.

1. Create a definition: Right-click → Create → TopsonGames → Unit Audio → `AudioDefinitionSO`. Assign several clips (one is picked at random), a volume, a pitch variance and a cooldown.
2. Make sure an **AudioManager** exists in the scene.
3. Add **UnitAudio** to your unit prefab and assign the definitions for footsteps, swings and death. Archers use `UnitAudioArcher`, which adds bow sounds.

All of these are triggered by the animation events described in 6.5, so a unit with no animation events will also be silent.

11.2 Formation flags

1. Make sure a **FlagManager** is in the scene – it rotates all flags towards the camera.
2. Create a flag prefab with the `FlagController` script. It can optionally show a health bar and status icons such as *Engaged* or *Flanked*.
3. Place the flag prefab as a child of the **FormationCollider** in your formation prefab.

11.3 Camera controller

An RTS camera is included. Add `CameraController` to your main camera, link the Input System actions for move, zoom and rotate, and assign the two **WorldCorner** transforms so the camera stays inside the map. Pan speed, zoom limits and screen-edge scrolling are configurable.

12. UI, minimap and siege HUD

12.1 Siege global UI (balance of power)

The bar at the top of the screen. The background represents the defender, the fill represents the attacker's score progress, and values are interpolated smoothly. Use an Image with *Image Type: Filled* and assign it to the SiegeGlobalUI script together with the SiegeManager.

12.2 Siege zone UI

Floating icons above capture zones. The colour is relative to the player – blue for friendly, red for hostile, regardless of who is attacker or defender – and a progress ring shows the current capture. Add SiegeZoneUI to a world-space canvas prefab and link it to a SiegeCaptureZone.

12.3 Minimap

MinimapController renders the map and needs the two WorldCorner transforms to know the bounds. Units, gates, towers and capture zones implement IMinimapTrackable and can supply their own sprite, so a tower shows a tower icon and a capture zone shows a flag. Icon colours update live when ownership changes.

13. Custom behaviour and API

LSCS is built so you can add new unit behaviours, tactics, weapons and overlays without editing a single core script. Every extension point is an abstract ScriptableObject: inherit, add [CreateAssetMenu], override the abstract methods.

13.1 Custom CombatBehaviourSO

```
using UnityEngine;

namespace TopsonGames
{
    [CreateAssetMenu(fileName = "MyCustomBehaviour",
                     menuName = "TopsonGames/Combat Behaviours/Custom")]
    public class MyCustomBehaviour : CombatBehaviourSO
    {
        public override void TickCombat(Unit unit, float deltaTime,
                                         Coroutine movementRoutine)
        {
            // your combat logic
        }
        // ... implement the remaining abstract methods
    }
}
```

The methods you will care about most:

Method	Called when
InitializeUnit(Unit, Formation)	Once per soldier on spawn. Randomise timers here so a formation does not attack in perfect unison.
InitializeFormation(Formation)	Once per formation on spawn. Set ammunition and volley timers here.

Method	Called when
TickIdle(...)	Every frame while the soldier is Idle. Usually walks it back to its waypoint.
TickMovement(...)	Every frame while the soldier is Moving. Usually checks for arrival.
TickCombat(...)	Every frame while the soldier is Fighting. Attack range, cooldowns and attack animations belong here.
OnMovementTick(Unit)	Continuously via coroutine while fighting. Ideal for repositioning – cavalry circling, infantry pushing in.
OnUpdateUnit / OnUpdateFormation	Per-frame hooks for animation blending and formation-wide logic such as archer volleys.
ShouldEngage / OnFindClosestEnemy	Target acquisition rules.
OnTakeDamage / OnDeath	Reaction hooks.
DrawGizmosOnBehaviour(Formation)	Editor visualisation.
OnReportArrows / OnReportArrowsUnit	Reaction to incoming arrows, e.g. raising shields.
IsFacingTarget(...)	Facing check. Also used by the combat rulesets to decide whether a hit is frontal.

Warning: A ScriptableObject instance is **shared by every soldier using it**. Never store per-soldier state in a field of your behaviour – put it on the Unit or Formation instead. This is the classic cause of "all my units attack at exactly the same moment".

13.2 Custom CombatPlacementSO

Override `TickUpdateEngagement(Formation formation, Formation target)`. It is called every frame for an engaged formation, and you implement flocking, line holding or envelopment by writing to `unit.Waypoint.transform.position` for each unit.

13.3 Custom AITacticSO

```
[CreateAssetMenu(fileName = "Tactic_MyTactic",
    menuName = "TopsonGames/AI/Tactic/My Custom Tactic")]
public class MyCustomTactic : AITacticSO
{
    public override float Evaluate(AICommander commander,
        List<Formation> allEnemies, List<Formation> availableFormations)
    {
        return 50f; // how good is this tactic right now?
    }

    public override void Execute(AICommander commander,
        List<Formation> allEnemies, List<Formation> availableFormations)
    {
        // issue orders, then:
        // commander.CommitFormationsToAction(...);
    }
}
```

Evaluate() analyses the battlefield and returns a score; the commander runs tactics in score order. Execute() issues the orders, typically by calling SetCustomTarget() on formations from availableFormations. **Always finish with `commander.CommitFormationsToAction()`** so the formations you used are marked busy and no other tactic grabs them.

13.4 Custom RangedWeaponSO

Override Attack(RangedWeapon rangedWeapon, Unit currentTarget, Formation targetFormation, Formation parentFormation). Pull a projectile with rangedWeapon.GetNextArrow(), spawn it at rangedWeapon.ShootPoint, set projectile.rigidBody.linearVelocity and assign projectile.parentFormation so kills are attributed correctly.

13.5 Custom VisualizerSO

Override TickVisualize(CombatBehaviourSO combatBehaviourSO, Formation formation, LineRenderer lineRenderer). It is called every frame the visualiser is active; compute your points and push them into the LineRenderer.

13.6 Formation.cs – useful members

Member	Purpose
List<Unit> GetUnits()	All living units of this formation.
Vector3 CalculateUnitCenter()	Average world position of the soldiers.
Quaternion CalculateAverageRotation()	Average facing.
void SetCustomTarget(Formation enemy)	Order this formation to attack a specific enemy formation.
void SetMoveOrder(bool moveToCustomEnemy = false)	Start a movement.
void Disengage()	Return from Engaged to Idle.
FormationState CurrentState	Idle, MovingToWaypoint, Engaged or Fleeing.

Member	Purpose
CombatState CurrentCombatState	Melee or Archer. Relevant for hybrid units that switch weapons.
Formation customEnemy	The enemy this formation was ordered to attack. Takes priority over targets of opportunity.
Formation ArcherTarget	The formation chosen for the next volley.
int numberOfUnits	Current living soldier count, updated automatically.
Transform WaypointCenter	Anchor of the formation grid. This is what travels along the NavMesh path.
Transform WaypointIndicator	Parent of the per-unit preview indicators used while dragging an order.
int currentArrows	Remaining volleys.
bool isFreeShooting	If true, archers shoot anything in range; if false, only their assigned customEnemy. Also toggled from the FormationUI.
bool isDefender	Siege role flag.
bool IsChargeBonusActive	Whether the charge damage bonus is currently running.
float currentMorale / maxMorale	Morale state.
ArmyData armyData	Link back to the ArmySO entry this formation was spawned from, including the troop count.

13.7 Unit.cs – useful members

Member	Purpose
DestructibleTarget GetCurrentTarget()	The enemy this soldier is fighting.
void SetCurrentTarget(DestructibleTarget)	Assign an opponent.
DestructibleTarget GetClosestEnemy()	Nearest enemy as computed by the job system.
void MoveTo(Vector3) / StopMovement()	Movement control through the NavMeshAgent.
UnitDataSO ActiveUnitData	specificUnitData if set, otherwise the formation's UnitData.
CombatBehaviourSO ActiveCombatBehaviour	The behaviour actually driving this soldier.
Vector3 lastTargetPosition	Last known target position. Use it in OnMovementTick to avoid re-issuing MoveTo every frame.
float attackTimer	Attack cooldown, counted down by TickCombat.

Member	Purpose
float findEnemyTimer	Scan cooldown for new enemies.
int rowInFormation	Which rank this soldier stands in, 1 being the front. Set automatically. Used for rank fire and kneeling.
int currentIdleAnimation	Index of the idle variant currently playing.
bool hasAppliedKnockback	Guard so one charge applies its knockback only once.
void ApplyKnockback(...)	Throw this soldier back.
UnityEvent OnDeath / OnDoDamage / OnReviveDamage	Inspector hooks for VFX, score, quests.

13.8 FormationController.cs

- `List<Formation> GetFormations()` – every formation in the scene
- `List<Formation> GetSelectedFormations()` – the current player selection
- `void AddSelectedFormation(Formation) / void ClearSelectedFormations()`
- `void SetGameMode(GameMode)` – switch between PlayerVsAI and Spectator_AIVsAI at runtime

14. Troubleshooting

The issues below cover the overwhelming majority of support requests. It is worth reading once even if nothing is broken yet.

Symptom	Likely cause and fix
Units play the attack animation but nobody dies	The animation has no OnAttack event, or <code>RelayAnimationEvents</code> is missing its Unit reference. For baked clips the event has to be added on the generated <code>MeshAnimation</code> asset, not on the original clip. See 6.5.
I cannot select my formations	Ground Mask and Unit Mask on the <code>FormationController</code> are the same layer, or the unit prefab is not on the Unit layer. Also check that the formation's Team ID matches the controller's Team ID.
<code>NullReferenceException</code> as soon as cavalry attacks	The combat behaviour and the <code>AnimatorLink</code> do not match. <code>CavalryCombatSO</code> requires an <code>AnimatorLinkCavalry</code> -derived link. See the matrix in 5.5.
The horse never animates, only the rider	You used a <i>Single</i> behaviour with a two-mesh link, or the horse's <code>MeshAnimator</code> slots (<code>HorseIdle</code> , <code>HorseWalk</code> , <code>HorseDeath</code>) are empty.
Units stand in front of a closed gate and do nothing	Gate is missing from <i>Attackable Target Types</i> in the <code>UnitDataSO</code> , or the <code>GameManager</code> 's <i>Building Attack Range Buffer</i> is too small. Try 1.5.
The AI does nothing at all	Either no strategy is assigned to the <code>AICommander</code> , or the tactics inside the strategy have empty <code>UnitTypeSO</code> arrays – a tactic with no eligible types scores 0 and never runs.

Symptom	Likely cause and fix
The AI walks in bizarre circles	BattleGrid danger values or <i>Pathfinding Clearance</i> are too high. Turn on the Danger gizmo to see what it is avoiding, or simply remove the BattleGridManager – everything works without it.
Ground troops attack enemies standing on a wall	Lower <i>Collider Height</i> in the UnitDataSO to about 3.0 and check <i>Max Engagement Height Difference</i> on the FormationCollider.
Formations do not detect each other	The parent formation has no kinematic Rigidbody , the collider is not set to <i>IsTrigger</i> , or the EngagementTrigger is missing or on the wrong layer.
Archers fire once and then stop	Ammunition (Arrows in the UnitDataSO) is exhausted, and the formation has switched to melee. That is intended – raise the value or <i>timeBetweenShots</i> .
All soldiers attack in perfect unison	Timers are not randomised. Randomise <i>attackTimer</i> in <i>InitializeUnit</i> , as the shipped behaviours do.
Fleeing units run forever	The scene has no MapExit.
Defenders ignore a wall entirely	That <i>DefensiveZone</i> is not registered in the SiegeManager's Wall Zones list, or its <i>Preferred Unit Types</i> exclude every unit you have.
Frame rate collapses with many units	Check that GPU Instancing is enabled on the character material, that you are actually using baked meshes at distance, and that the material count per unit is 1.

15. Component reference

15.1 Managers and controllers

Component	Description	Required
GameManager	Spawns armies, evaluates win conditions, owns shared job data, computes army strength.	Yes
FormationController	Player input, selection, orders, highlighting, game mode.	Yes
CameraController	RTS camera movement, zoom and rotation.	Yes
TargetManager	Maintains fast lookup lists of all attackable targets.	Yes
UIManager	All in-game UI except the minimap.	Recommended
SiegeManager	Gates, capture zones, attacker/defender roles, siege win conditions.	Siege maps
BattleGridManager	Influence maps for the AI. Field battles only.	Optional
AICommander	One per AI-controlled army.	Per army

Component	Description	Required
MobileInputManager	Touch controls.	Mobile
MinimapController	Renders the minimap.	Optional
AudioManager	Pooled audio playback.	Optional
FlagManager	Rotates formation flags towards the camera.	Optional
MeshInstancingManager	GPU instancing and LOD for baked meshes.	Baked meshes
AnimatorLinkLODManager	Switches units between Animator and baked mesh.	Hybrid mode
AnimatorLODManager	Throttles Animator update rate by distance.	Optional

15.2 Formation and movement

Component	Description
Formation	Manages a group of units: state, waypoints, morale, grid arrangement, orders.
FormationCollider	The formation's "eyes". Detects enemy engagement triggers and starts combat.
EngagementTrigger	The formation's "body". What other formations detect.
CohesionJob	Burst job adjusting individual speeds based on distance to the waypoint.
FindClosestTargetJob	Burst job finding the nearest enemy unit or building surface per soldier.
FindClosestEnemyJob	Burst job computing the nearest enemy per unit within a formation.
RotationJob	Burst job computing all unit rotations in parallel.
SkirmisherKiteAgent	Added automatically by the harass tactic. Keeps a mounted skirmisher at standoff range and rides away from approaching enemies.

15.3 Unit and combat

Component / asset	Description
Unit	A single soldier. Health, state and references. Acts as a middleman for the behaviours.
DestructibleTarget	Required on everything attackable – units, gates, walls. Handles HP, Team ID and death events.
UnitDataSO	Complete stat and behaviour blueprint of a unit type.
UnitTypeSO	Category label used by the damage table, the AI and the zones.
CombatBehaviourSO	Base class of all soldier behaviours.
MeleeCombatSO	Standard infantry.

Component / asset	Description
ArcherCombatSO	Foot archers, volley fire, falls back to melee.
MusketeerCombatSO	Line infantry with rank fire and kneeling front row.
CavalryCombatSO	Cavalry with separate horse mesh and animator.
CavalrySingleCombatSO	Cavalry baked as one single mesh.
HorseArcherCombatSO	Mounted archers, two meshes, 360° fire.
HorseArcherSingleCombatSO	Mounted archers, single mesh, 360° fire.
ElephantCombatSO	Elephants with area knockback. Also a worked extension example.
CombatPlacementSO / MeleeCombatPlacementSO	Boids-based formation placement during combat.
CombatRulesetSO	Base class for damage models.
StandardCombatRulesetSO	Modern HP system with elevation and flanking multipliers.
RomeOneHPCombatRulesetSO	Rome 1 style two-roll connect/lethality system.
DamageModifierSO	Rock-paper-scissors table, armour maths and the active ruleset.
RangedWeapon	Component on the weapon model. Owns the shoot point and the projectile pool.
RangedWeaponSO / IBowSO / MusketSO	Trajectory definitions for bows and firearms.
Projectile	Pooled projectile with lifetime and delayed collider activation.
EffectSO	A status a formation can carry, feeding morale and the UI.

15.4 Animation

Component	Description
AnimatorLink	Bridge between the combat logic and the animation backend. Lets the system swap between Unity's Animator and the baked mesh system.
AnimatorLinkCavalry	Animator link with a second animator for the mount.
AnimatorLinkMesh	Baked-mesh link for infantry and foot archers.
AnimatorLinkMeshMusketeer	Adds the kneeling fire clip and rank awareness.
AnimatorLinkMeshCavalry	Baked-mesh link with separate rider and horse MeshAnimators.
AnimatorLinkMeshCavalrySingle	Baked-mesh link for rider and horse in one mesh.
AnimatorLinkMeshHorseArchers	Two-mesh horse archer link with eight directional firing clips.

Component	Description
AnimatorLinkMeshHorseArchersSingle	Single-mesh horse archer link with separate idle and riding firing sets.
RelayAnimationEvents	Forwards Unity animation events to the Unit and UnitAudio.
RelayAnimationEventsMesh	The same for baked mesh animation events.
MeshAnimator	Plays baked clips, manages LOD levels and bone lookups for attachments.
MeshAnimation	A baked clip: frames, frame rate, looping and events.
AttachmentPoint	Attaches a weapon to a baked bone.
AnimatorLOD / AnimatorLODSettings	Per-unit data for Animator update-rate throttling.

15.5 AI

Component / asset	Description
AICommander	The AI brain. Evaluates and executes tactics.
AIStrategySO	A named list of tactics forming an AI personality.
AITacticSO	Base class for a single manoeuvre.
Tactic_ArmyBattleLine	Deploys and charges all melee formations as one line.
Tactic_ReserveFlankGuard	Keeps a reserve and answers flank and rear attacks.
Tactic_EstablishFrontLine	Older per-formation front line matching.
Tactic_FlankWithCavalry	Sends cavalry around the clash onto vulnerable targets.
Tactic_HorseArcherHarass	Kiting behaviour for mounted skirmishers.
Tactic_ArcherTargetSelection	Picks targets and firing positions for ranged formations.
Tactic_AttackGate	Attacker: picks and breaches the best gate.
Tactic_OccupyArcherPositions	Attacker: occupies firing positions outside the walls.
Tactic_ManTheWalls	Defender: assigns archers to walls by dynamic priority.
Tactic_DefendGate	Defender: blocks threatened gates with melee.
Tactic_CapturePoints	Attacker: captures victory points after a breach.
Tactic_RecapturePoints	Defender: retakes lost victory points.
BattleGridManager / BattleGrid	Influence maps, safe pathfinding and debug gizmos. Optional, field battles only.
BattleGridUnitSettingsSO	Per-unit-type perception tuning for the grid.

15.6 Army, siege and world

Component / asset	Description
ArmySO / ArmyData	Army roster: unit types, troop counts and the spawned formation reference.
ArmySpawner	Spawns an army across its spawn points.
SpawnPoint	One spawn position with allowed unit types and facing correction.
GateController	Destructible gate: NavMeshObstacle, visual swap, defender and attacker counting.
SiegeCaptureZone	Victory point with capture timer, priority, choke points and events.
DefensiveZone	Defender standing position with base and dynamic priority.
AttackerZone	Attacker staging or firing position.
ThreatSensor	Trigger volume counting enemies, feeding dynamic priorities.
SiegeChokePoint	Valid entrance into a capture zone with a capacity limit.
StairTrigger	Level change between ground and wall.
TowerController	Automatic defensive tower with range, damage and cooldown.
MapExit	Where routing formations leave the battlefield.
NoPlacementZone	Blocks formation placement on the attached collider.

15.7 UI and visualisation

Component	Description
MinimapController / MinimapIcon	Renders the minimap and its live-coloured icons.
FormationUI	Unit card per formation; clicking it selects the formation.
HoverUI	Stat panel shown on hover.
SiegeGlobalUI	The balance-of-power bar.
SiegeZoneUI	Floating world-space icon above a capture zone.
FlagManager / FlagController	Formation flags with health bars and status icons.
Visualizer / VisualizerSO	LineRenderer-based tactical overlays: paths, detection areas, aura radii.
UnitAudio / UnitAudioArcher / AudioDefinitionSO / AudioManager	The pooled audio system.

Thank you for using the Large Scale Combat System. If something in this document is unclear or out of date, please tell us on Discord – documentation feedback is genuinely welcome.